

File Tree

해시 확인: certutil -hashfile <dist> SHA256

dist/

```
├─ assets/
│   └─ ai.worker-B7h8TIdl.js (46508 B / 567a2ecbdf25)
│   └─ index-BVkn0d8S.css (10778 B / 969ee06a92c7)
│   └─ index-CSn2cQoE.js (63328 B / 21f79ae3982c)
└─ index.html (366 B / 3a75234ac1db)
```

==== assets/ai.worker-B7h8TIdl.js (46508 B / 567a2ecbdf25) ====

```
1 | |(function () {
2 | |   'use strict';
3 | |   const MASK64 = 0xffffffffffffffff;
4 | |   function rotl(x, k) {
5 | |     return 0xffffffffffffffff & ((x << k) | (x >> (64n - k)));
6 | |   }
7 | |   function wrappingMul(x, y) {
8 | |     return (x * y) & MASK64;
9 | |   }
10 | |   const rand = (function xoroshiro128(state) {
11 | |     return function () {
12 | |       let s0 = BigInt(state & MASK64),
13 | |       s1 = BigInt((state >> 64n) & MASK64);
14 | |       const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
15 | |       return (
16 | |         (s1 ^ s0),
17 | |         (s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64),
18 | |         (s1 = rotl(s1, 37n)),
19 | |         (state = (s1 << 64n) | s0),
20 | |         result
21 | |       );
22 | |     };
23 | |   })(0xa187eb39cdcaed8f31c4b365b102e01en),
24 | |   PIECE_KEYS = Array.from({ length: 2 }, () =>
25 | |     Array.from({ length: 6 }, () => Array.from({ length: 128 }, () => rand()))),
26 | |   ,
27 | |   EP_KEYS = Array.from({ length: 8 }, () => rand()),
28 | |   CASTLING_KEYS = Array.from({ length: 16 }, () => rand()),
29 | |   SIDE_KEY = rand(),
30 | |   DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1';
31 | |   class Move {
32 | |     color;
33 | |     from;
34 | |     to;
35 | |     piece;
36 | |     captured;
37 | |     promotion;
38 | |     flags;
39 | |     san;
40 | |     lan;
41 | |     before;
42 | |     after;
43 | |     constructor(chess, internal) {
44 | |       const {
45 | |         color: color,
46 | |         piece: piece,
47 | |         from: from,
48 | |         to: to,
49 | |         flags: flags,
50 | |         captured: captured,
51 | |         promotion: promotion,
52 | |       } = internal,
53 | |       fromAlgebraic = algebraic(from),
54 | |       toAlgebraic = algebraic(to);
55 | |       ((this.color = color),
56 | |       (this.piece = piece),
57 | |       (this.from = fromAlgebraic),
58 | |       (this.to = toAlgebraic),
59 | |       (this.san = chess._moveToSan(internal, chess._moves({ legal: !0 }))),
```

```

60 |         (this.lan += fromAlgebraic + toAlgebraic),
61 |         (this.before = chess.fen()),
62 |         chess._makeMove(internal),
63 |         (this.after = chess.fen()),
64 |         chess._undoMove(),
65 |         (this.flags = '');
66 |         for (const flag in BITS) BITS[flag] & flags && (this.flags += FLAGS[flag]);
67 |         (captured && (this.captured = captured),
68 |         promotion && ((this.promotion = promotion), (this.lan += promotion)));
69 |     }
70 |     isCapture() {
71 |         return this.flags.indexOf(FLAGS.CAPTURE) > -1;
72 |     }
73 |     isPromotion() {
74 |         return this.flags.indexOf(FLAGS.PROMOTION) > -1;
75 |     }
76 |     isEnPassant() {
77 |         return this.flags.indexOf(FLAGS.EP_CAPTURE) > -1;
78 |     }
79 |     isKingsideCastle() {
80 |         return this.flags.indexOf(FLAGS.KSIDE_CASTLE) > -1;
81 |     }
82 |     isQueensideCastle() {
83 |         return this.flags.indexOf(FLAGS.QSIDE_CASTLE) > -1;
84 |     }
85 |     isBigPawn() {
86 |         return this.flags.indexOf(FLAGS.BIG_PAWN) > -1;
87 |     }
88 | }
89 | const FLAGS = {
90 |     NORMAL: 'n',
91 |     CAPTURE: 'c',
92 |     BIG_PAWN: 'b',
93 |     EP_CAPTURE: 'e',
94 |     PROMOTION: 'p',
95 |     KSIDE_CASTLE: 'k',
96 |     QSIDE_CASTLE: 'q',
97 |     NULL_MOVE: '-',
98 | },
99 | BITS = {
100 |     NORMAL: 1,
101 |     CAPTURE: 2,
102 |     BIG_PAWN: 4,
103 |     EP_CAPTURE: 8,
104 |     PROMOTION: 16,
105 |     KSIDE_CASTLE: 32,
106 |     QSIDE_CASTLE: 64,
107 |     NULL_MOVE: 128,
108 | },
109 | 0x88 = {
110 |     a8: 0,
111 |     b8: 1,
112 |     c8: 2,
113 |     d8: 3,
114 |     e8: 4,
115 |     f8: 5,
116 |     g8: 6,
117 |     h8: 7,
118 |     a7: 16,
119 |     b7: 17,
120 |     c7: 18,
121 |     d7: 19,
122 |     e7: 20,
123 |     f7: 21,
124 |     g7: 22,
125 |     h7: 23,
126 |     a6: 32,
127 |     b6: 33,
128 |     c6: 34,
129 |     d6: 35,
130 |     e6: 36,

```

```

131 | | | f6: 37,
132 | | | g6: 38,
133 | | | h6: 39,
134 | | | a5: 48,
135 | | | b5: 49,
136 | | | c5: 50,
137 | | | d5: 51,
138 | | | e5: 52,
139 | | | f5: 53,
140 | | | g5: 54,
141 | | | h5: 55,
142 | | | a4: 64,
143 | | | b4: 65,
144 | | | c4: 66,
145 | | | d4: 67,
146 | | | e4: 68,
147 | | | f4: 69,
148 | | | g4: 70,
149 | | | h4: 71,
150 | | | a3: 80,
151 | | | b3: 81,
152 | | | c3: 82,
153 | | | d3: 83,
154 | | | e3: 84,
155 | | | f3: 85,
156 | | | g3: 86,
157 | | | h3: 87,
158 | | | a2: 96,
159 | | | b2: 97,
160 | | | c2: 98,
161 | | | d2: 99,
162 | | | e2: 100,
163 | | | f2: 101,
164 | | | g2: 102,
165 | | | h2: 103,
166 | | | a1: 112,
167 | | | b1: 113,
168 | | | c1: 114,
169 | | | d1: 115,
170 | | | e1: 116,
171 | | | f1: 117,
172 | | | g1: 118,
173 | | | h1: 119,
174 | | | },
175 | | | PAWN_OFFSETS = { b: [16, 32, 17, 15], w: [-16, -32, -17, -15] },
176 | | | PIECE_OFFSETS = {
177 | | |   n: [-18, -33, -31, -14, 18, 33, 31, 14],
178 | | |   b: [-17, -15, 17, 15],
179 | | |   r: [-16, 1, 16, -1],
180 | | |   q: [-17, -16, -15, 1, 17, 16, 15, -1],
181 | | |   k: [-17, -16, -15, 1, 17, 16, 15, -1],
182 | | | },
183 | | | ATTACKS = [
184 | | |   20, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0,
185 | | |   20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0,
186 | | |   0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 20, 2,
187 | | |   24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 24, 24, 24,
188 | | |   24, 24, 24, 56, 0, 56, 24, 24, 24, 24, 24, 24, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0,
189 | | |   0, 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20,
190 | | |   0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0,
191 | | |   0, 0, 0, 20, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0,
192 | | |   24, 0, 0, 0, 0, 0, 0, 20,
193 | | | ],
194 | | | RAYS = [
195 | | |   17, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0,
196 | | |   15, 0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15, 0, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0,
197 | | |   0, 15, 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 17, 0,
198 | | |   16, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
199 | | |   1, 1, 0, -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0, 0, 0,
200 | | |   0, 0, 0, 0, 0, -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0, 0, -17,
201 | | |   0, 0, 0, 0, 0, 0, 0, -15, 0, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, 0, -15, 0, 0, 0, 0,

```

```

202 | | | -16, 0, 0, 0, 0, -17, 0, 0, 0, 0, -15, 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, -17, 0, 0, -15, 0,
203 | | | 0, 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, 0, 0, -17,
204 | | | ],
205 | | | PIECE_MASKS = { p: 1, n: 2, b: 4, r: 8, q: 16, k: 32 },
206 | | | PROMOTIONS = [ 'n', 'b', 'r', 'q' ],
207 | | | ROOKS = {
208 | | |   w: [
209 | | |     { square: 0x88.a1, flag: BITS.QSIDE_CASTLE },
210 | | |     { square: 0x88.h1, flag: BITS.KSIDE_CASTLE },
211 | | |   ],
212 | | |   b: [
213 | | |     { square: 0x88.a8, flag: BITS.QSIDE_CASTLE },
214 | | |     { square: 0x88.h8, flag: BITS.KSIDE_CASTLE },
215 | | |   ],
216 | | | },
217 | | | SECOND_RANK = { b: 1, w: 6 };
218 | | | function rank(square) {
219 | | |   return square >> 4;
220 | | | }
221 | | | function file(square) {
222 | | |   return 15 & square;
223 | | | }
224 | | | function isDigit(c) {
225 | | |   return -1 <= '0123456789'.indexOf(c);
226 | | | }
227 | | | function algebraic(square) {
228 | | |   const f = file(square),
229 | | |         r = rank(square);
230 | | |   return 'abcdefgh'.substring(f, f+1) + '87654321'.substring(r, r+1);
231 | | | }
232 | | | function swapColor(color) {
233 | | |   return 'w' === color ? 'b' : 'w';
234 | | | }
235 | | | function addMove(moves, color, from, to, piece, captured = void 0, flags = BITS.NORMAL) {
236 | | |   const r = rank(to);
237 | | |   if ('p' !== piece || (7 <= r & 0 <= r))
238 | | |     moves.push({
239 | | |       color: color,
240 | | |       from: from,
241 | | |       to: to,
242 | | |       piece: piece,
243 | | |       captured: captured,
244 | | |       flags: flags,
245 | | |     });
246 | | |   else
247 | | |     for (let i = 0; i < PROMOTIONS.length; i++) {
248 | | |       const promotion = PROMOTIONS[i];
249 | | |       moves.push({
250 | | |         color: color,
251 | | |         from: from,
252 | | |         to: to,
253 | | |         piece: piece,
254 | | |         captured: captured,
255 | | |         promotion: promotion,
256 | | |         flags: flags | BITS.PROMOTION,
257 | | |       });
258 | | |     }
259 | | | }
260 | | | class Chess {
261 | | |   _board = new Array(128);
262 | | |   _turn = 'w';
263 | | |   _kings = { w: -1, b: -1 };
264 | | |   _epSquare = -1;
265 | | |   _halfMoves = 0;
266 | | |   _moveNumber = 0;
267 | | |   _history = [];
268 | | |   _castling = { w: 0, b: 0 };
269 | | |   _hash = 0n;
270 | | |   _positionCount = new Map();
271 | | |   constructor(fen = DEFAULT_POSITION, { skipValidation: skipValidation = !1 } = {}) {
272 | | |     this.load(fen, { skipValidation: skipValidation });

```

```

273 |     }
274 |     clear() {
275 |         ((this._board = new Array(128)),
276 |         (this._kings = {w: -1, b: -1}),
277 |         (this._turn = 'w'),
278 |         (this._castling = {w: 0, b: 0}),
279 |         (this._epSquare = -1),
280 |         (this._halfMoves = 0),
281 |         (this._moveNumber = 1),
282 |         (this._history = []),
283 |         (this._hash = this._computeHash()),
284 |         (this._positionCount = new Map()));
285 |     }
286 |     load(fen, { skipValidation: skipValidation !== 1 } = {}) {
287 |         let tokens = fen.split(/\s+/);
288 |         if (tokens.length ≥ 2 && tokens.length < 6) {
289 |             const adjustments = ['-', '-', '0', '1'];
290 |             fen = tokens.concat(adjustments.slice(-(6 - tokens.length))).join(' ');
291 |         }
292 |         if (((tokens = fen.split(/\s+/)), !skipValidation)) {
293 |             const { ok: ok, error: error } = (function validateFen(fen) {
294 |                 const tokens = fen.split(/\s+/);
295 |                 if (6 !== tokens.length)
296 |                     return { ok: !1, error: 'Invalid FEN: must contain six space-delimited fields' };
297 |                 const moveNumber = parseInt(tokens[5], 10);
298 |                 if (isNaN(moveNumber) || moveNumber ≤ 0)
299 |                     return { ok: !1, error: 'Invalid FEN: move number must be a positive integer' };
300 |                 const halfMoves = parseInt(tokens[4], 10);
301 |                 if (isNaN(halfMoves) || halfMoves < 0)
302 |                     return {
303 |                         ok: !1,
304 |                         error: 'Invalid FEN: half move counter number must be a non-negative integer',
305 |                     };
306 |                 if (!/^(?![abcdefgh][36])$/ .test(tokens[3]))
307 |                     return { ok: !1, error: 'Invalid FEN: en-passant square is invalid' };
308 |                 if (/^[^kKqQ-]/ .test(tokens[2]))
309 |                     return { ok: !1, error: 'Invalid FEN: castling availability is invalid' };
310 |                 if (!/^(w|b)$/ .test(tokens[1]))
311 |                     return { ok: !1, error: 'Invalid FEN: side-to-move is invalid' };
312 |                 const rows = tokens[0].split('/');
313 |                 if (8 !== rows.length)
314 |                     return {
315 |                         ok: !1,
316 |                         error: 'Invalid FEN: piece data does not contain 8 '/'-delimited rows',
317 |                     };
318 |                 for (let i = 0; i < rows.length; i++) {
319 |                     let sumFields = 0,
320 |                         previousWasNumber = !1;
321 |                     for (let k = 0; k < rows[i].length; k++)
322 |                         if (isDigit(rows[i][k])) {
323 |                             if (previousWasNumber)
324 |                                 return {
325 |                                     ok: !1,
326 |                                     error: 'Invalid FEN: piece data is invalid (consecutive number)',
327 |                                 };
328 |                             ((sumFields += parseInt(rows[i][k], 10)), (previousWasNumber = !0));
329 |                         } else {
330 |                             if (!/^[prnbqkPRNBQK]$/ .test(rows[i][k]))
331 |                                 return { ok: !1, error: 'Invalid FEN: piece data is invalid (invalid piece)' };
332 |                             ((sumFields += 1), (previousWasNumber = !1));
333 |                         }
334 |                 if (8 !== sumFields)
335 |                     return {
336 |                         ok: !1,
337 |                         error: 'Invalid FEN: piece data is invalid (too many squares in rank)',
338 |                     };
339 |             }
340 |             if (
341 |                 ('3' === tokens[3][1] && 'w' === tokens[1]) ||
342 |                 ('6' === tokens[3][1] && 'b' === tokens[1])
343 |             )

```

```

344 |         return { ok: !1, error: 'Invalid FEN: illegal en-passant square' };
345 |         const kings = [
346 |             { color: 'white', regex: /K/g },
347 |             { color: 'black', regex: /k/g },
348 |         ];
349 |         for (const { color, regex } of kings) {
350 |             if (!regex.test(tokens[0]))
351 |                 return { ok: !1, error: `Invalid FEN: missing ${color} king` };
352 |             if ((tokens[0].match(regex) || []).length > 1)
353 |                 return { ok: !1, error: `Invalid FEN: too many ${color} kings` };
354 |         }
355 |         return Array.from(rows[0] + rows[7]).some((char) => 'P' === char.toUpperCase())
356 |             ? { ok: !1, error: 'Invalid FEN: some pawns are on the edge rows' }
357 |             : { ok: !0 };
358 |     })(fen);
359 |     if (!ok) throw new Error(error);
360 | }
361 | const position = tokens[0];
362 | let square = 0;
363 | this.clear();
364 | for (let i = 0; i < position.length; i++) {
365 |     const piece = position.charAt(i);
366 |     if ('/' === piece) square += 8;
367 |     else if (isDigit(piece)) square += parseInt(piece, 10);
368 |     else {
369 |         const color = piece < 'a' ? 'w' : 'b';
370 |         (this._put({ type: piece.toLowerCase(), color: color }, algebraic(square)), square++);
371 |     }
372 | }
373 | ((this._turn = tokens[1],
374 | tokens[2].indexOf('K') > -1 && (this._castling.w |= BITS.KSIDE_CASTLE),
375 | tokens[2].indexOf('Q') > -1 && (this._castling.w |= BITS.QSIDE_CASTLE),
376 | tokens[2].indexOf('k') > -1 && (this._castling.b |= BITS.KSIDE_CASTLE),
377 | tokens[2].indexOf('q') > -1 && (this._castling.b |= BITS.QSIDE_CASTLE),
378 | (this._epSquare = '-' === tokens[3] ? -1 : 0x88[tokens[3]]),
379 | (this._halfMoves = parseInt(tokens[4], 10)),
380 | (this._moveNumber = parseInt(tokens[5], 10)),
381 | (this._hash = this._computeHash()),
382 | this._incPositionCount());
383 | }
384 | fen({ forceEnpassantSquare: forceEnpassantSquare = !1 } = {}) {
385 |     let empty = 0,
386 |         fen = '';
387 |     for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
388 |         if (this._board[i]) {
389 |             empty > 0 && ((fen += empty), (empty = 0));
390 |             const { color, type: piece } = this._board[i];
391 |             fen += 'w' === color ? piece.toUpperCase() : piece.toLowerCase();
392 |         } else empty++;
393 |         (i + 1) & 136 &&
394 |             (empty > 0 && (fen += empty), i ≡ 0x88.h1 && (fen += '/'), (empty = 0), (i += 8));
395 |     }
396 |     let castling = '';
397 |     (this._castling.w & BITS.KSIDE_CASTLE && (castling += 'K'),
398 |     this._castling.w & BITS.QSIDE_CASTLE && (castling += 'Q'),
399 |     this._castling.b & BITS.KSIDE_CASTLE && (castling += 'k'),
400 |     this._castling.b & BITS.QSIDE_CASTLE && (castling += 'q'),
401 |     (castling = castling || '-'));
402 |     let epSquare = '-';
403 |     if (-1 ≡ this._epSquare)
404 |         if (forceEnpassantSquare) epSquare = algebraic(this._epSquare);
405 |     else {
406 |         const bigPawnSquare = this._epSquare + ('w' === this._turn ? 16 : -16),
407 |             squares = [bigPawnSquare + 1, bigPawnSquare - 1];
408 |         for (const square of squares) {
409 |             if (136 & square) continue;
410 |             const color = this._turn;
411 |             if (this._board[square]?.color === color && 'p' === this._board[square].type) {
412 |                 this._makeMove({
413 |                     color: color,
414 |                     from: square,

```

```

415 |         to: this._epSquare,
416 |         piece: 'p',
417 |         captured: 'p',
418 |         flags: BITS.EP_CAPTURE,
419 |     });
420 |     const isLegal = !this._isKingAttacked(color);
421 |     if ((this._undoMove(), isLegal)) {
422 |         epSquare = algebraic(this._epSquare);
423 |         break;
424 |     }
425 | }
426 | }
427 | }
428 | return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
429 | }
430 | _pieceKey(i) {
431 |     if (!this._board[i]) return 0n;
432 |     const { color, type } = this._board[i],
433 |           typeIndex = { p: 0, n: 1, b: 2, r: 3, q: 4, k: 5 }[type];
434 |     return PIECE_KEYS[{ w: 0, b: 1 }[color]][typeIndex][i];
435 | }
436 | _epKey() {
437 |     return -1 === this._epSquare ? 0n : EP_KEYS[7 & this._epSquare];
438 | }
439 | _castlingKey() {
440 |     const index = (this._castling.w >> 5) | (this._castling.b >> 3);
441 |     return CASTLING_KEYS[index];
442 | }
443 | _computeHash() {
444 |     let hash = 0n;
445 |     for (let i = 0x88.a8; i <= 0x88.h1; i++)
446 |         136 & i ? (i += 7) : this._board[i] && (hash ^= this._pieceKey(i));
447 |     return (
448 |         (hash ^= this._epKey()),
449 |         (hash ^= this._castlingKey()),
450 |         'b' === this._turn && (hash ^= SIDE_KEY),
451 |         hash
452 |     );
453 | }
454 | reset() {
455 |     this.load(DEFAULT_POSITION);
456 | }
457 | get(square) {
458 |     return this._board[0x88[square]];
459 | }
460 | _set(sq, piece) {
461 |     ((this._hash ^= this._pieceKey(sq)),
462 |     (this._board[sq] = piece),
463 |     (this._hash ^= this._pieceKey(sq)));
464 | }
465 | _put({ type, color }, square) {
466 |     if (-1 === 'pnbrqkPNBRQK'.indexOf(type.toLowerCase())) return !1;
467 |     if (!(square in 0x88)) return !1;
468 |     const sq = 0x88[square];
469 |     if ('k' === type && -1 !== this._kings[color] && this._kings[color] !== sq) return !1;
470 |     const currentPieceOnSquare = this._board[sq];
471 |     return (
472 |         currentPieceOnSquare &&
473 |         'k' === currentPieceOnSquare.type &&
474 |         (this._kings[currentPieceOnSquare.color] = -1),
475 |         this._set(sq, { type, color }),
476 |         'k' === type && (this._kings[color] = sq),
477 |         !0
478 |     );
479 | }
480 | _clear(sq) {
481 |     ((this._hash ^= this._pieceKey(sq)), delete this._board[sq]);
482 | }
483 | _attacked(color, square, verbose) {
484 |     const attackers = [];
485 |     for (let i = 0x88.a8; i <= 0x88.h1; i++) {

```

```

486 |         if (136 & i) {
487 |             i += 7;
488 |             continue;
489 |         }
490 |         if (void 0 === this._board[i] || this._board[i].color !== color) continue;
491 |         const piece = this._board[i],
492 |               difference = i - square;
493 |         if (0 === difference) continue;
494 |         const index = difference + 119;
495 |         if (ATTACKS[index] & PIECE_MASKS[piece.type]) {
496 |             if ('p' === piece.type) {
497 |                 if (
498 |                     (difference > 0 & 'w' === piece.color) ||
499 |                     (difference ≤ 0 & 'b' === piece.color)
500 |                 ) {
501 |                     if (!verbose) return !0;
502 |                     attackers.push(algebraic(i));
503 |                 }
504 |                 continue;
505 |             }
506 |             if ('n' === piece.type || 'k' === piece.type) {
507 |                 if (verbose) {
508 |                     attackers.push(algebraic(i));
509 |                     continue;
510 |                 }
511 |                 return !0;
512 |             }
513 |             const offset = RAYS[index];
514 |             let j = i + offset,
515 |                 blocked = !1;
516 |             for (; j !== square; ) {
517 |                 if (null !== this._board[j]) {
518 |                     blocked = !0;
519 |                     break;
520 |                 }
521 |                 j += offset;
522 |             }
523 |             if (!blocked) {
524 |                 if (verbose) {
525 |                     attackers.push(algebraic(i));
526 |                     continue;
527 |                 }
528 |                 return !0;
529 |             }
530 |         }
531 |     }
532 |     return !!verbose & attackers;
533 | }
534 | attackers(square, attackedBy) {
535 |     return attackedBy
536 |     ? this._attacked(attackedBy, 0x88[square], !0)
537 |     : this._attacked(this._turn, 0x88[square], !0);
538 | }
539 | _isKingAttacked(color) {
540 |     const square = this._kings[color];
541 |     return -1 !== square & this._attacked(swapColor(color), square);
542 | }
543 | hash() {
544 |     return this._hash.toString(16);
545 | }
546 | isAttacked(square, attackedBy) {
547 |     return this._attacked(attackedBy, 0x88[square]);
548 | }
549 | isCheck() {
550 |     return this._isKingAttacked(this._turn);
551 | }
552 | inCheck() {
553 |     return this.isCheck();
554 | }
555 | isCheckmate() {
556 |     return this.isCheck() & 0 === this._moves().length;

```



```

557 |     }
558 |     isStalemate() {
559 |         return !this.isCheck() && 0 === this._moves().length;
560 |     }
561 |     isInsufficientMaterial() {
562 |         const pieces = { b: 0, n: 0, r: 0, q: 0, k: 0, p: 0 },
563 |             bishops = [];
564 |         let numPieces = 0,
565 |             squareColor = 0;
566 |         for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
567 |             if (((squareColor = (squareColor + 1) % 2), 136 & i)) {
568 |                 i += 7;
569 |                 continue;
570 |             }
571 |             const piece = this._board[i];
572 |             piece &&
573 |                 ((pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1),
574 |                 'b' === piece.type && bishops.push(squareColor),
575 |                 numPieces++);
576 |         }
577 |         if (2 === numPieces) return !0;
578 |         if (3 === numPieces && (1 === pieces.b || 1 === pieces.n)) return !0;
579 |         if (numPieces === pieces.b + 2) {
580 |             let sum = 0;
581 |             const len = bishops.length;
582 |             for (let i = 0; i < len; i++) sum += bishops[i];
583 |             if (0 === sum || sum === len) return !0;
584 |         }
585 |         return !1;
586 |     }
587 |     isThreefoldRepetition() {
588 |         return this._getPositionCount(this._hash) ≥ 3;
589 |     }
590 |     isDrawByFiftyMoves() {
591 |         return this._halfMoves ≥ 100;
592 |     }
593 |     isDraw() {
594 |         return (
595 |             this.isDrawByFiftyMoves() ||
596 |             this.isStalemate() ||
597 |             this.isInsufficientMaterial() ||
598 |             this.isThreefoldRepetition()
599 |         );
600 |     }
601 |     isGameOver() {
602 |         return this.isCheckmate() || this.isDraw();
603 |     }
604 |     moves({ verbose: verbose = !1, square: square, piece: piece } = {}) {
605 |         const moves = this._moves({ square: square, piece: piece });
606 |         return verbose
607 |             ? moves.map((move) ⇒ new Move(this, move))
608 |             : moves.map((move) ⇒ this._moveToSan(move, moves));
609 |     }
610 |     _moves({ legal: legal = !0, piece: piece, square: square } = {}) {
611 |         const forSquare = square ? square.toLowerCase() : void 0,
612 |             forPiece = piece ? piece.toLowerCase(),
613 |             moves = [],
614 |             us = this._turn,
615 |             them = swapColor(us);
616 |         let firstSquare = 0x88.a8,
617 |             lastSquare = 0x88.h1,
618 |             singleSquare = !1;
619 |         if (forSquare) {
620 |             if (!(forSquare in 0x88)) return [];
621 |             ((firstSquare = lastSquare = 0x88[forSquare]), (singleSquare = !0));
622 |         }
623 |         for (let from = firstSquare; from ≤ lastSquare; from++) {
624 |             if (136 & from) {
625 |                 from += 7;
626 |                 continue;
627 |             }

```

```

628 |         if (!this._board[from] || this._board[from].color === them) continue;
629 |         const { type: type } = this._board[from];
630 |         let to;
631 |         if ('p' === type) {
632 |             if (forPiece && forPiece !== type) continue;
633 |             ((to = from + PAWN_OFFSETS[us][0]),
634 |             this._board[to] ||
635 |             (addMove(moves, us, from, to, 'p'),
636 |             (to = from + PAWN_OFFSETS[us][1]),
637 |             SECOND_RANK[us] !== rank(from) ||
638 |             this._board[to] ||
639 |             addMove(moves, us, from, to, 'p', void 0, BITS.BIG_PAWN)));
640 |             for (let j = 2; j < 4; j++)
641 |                 ((to = from + PAWN_OFFSETS[us][j]),
642 |                 136 & to ||
643 |                 (this._board[to]?.color === them
644 |                 ? addMove(moves, us, from, to, 'p', this._board[to].type, BITS.CAPTURE)
645 |                 : to === this._epSquare &&
646 |                 addMove(moves, us, from, to, 'p', 'p', BITS.EP_CAPTURE)));
647 |         } else {
648 |             if (forPiece && forPiece !== type) continue;
649 |             for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++) {
650 |                 const offset = PIECE_OFFSETS[type][j];
651 |                 for (to = from; (to += offset), !(136 & to);) {
652 |                     if (this._board[to]) {
653 |                         if (this._board[to].color === us) break;
654 |                         addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
655 |                         break;
656 |                     }
657 |                     if ((addMove(moves, us, from, to, type), 'n' === type || 'k' === type)) break;
658 |                 }
659 |             }
660 |         }
661 |     }
662 |     if (
663 |         !(
664 |             (void 0 !== forPiece && 'k' !== forPiece) ||
665 |             (singleSquare && lastSquare !== this._kings[us])
666 |         )
667 |     ) {
668 |         if (this._castling[us] & BITS.KSIDE_CASTLE) {
669 |             const castlingFrom = this._kings[us],
670 |                 castlingTo = castlingFrom + 2;
671 |             this._board[castlingFrom + 1] ||
672 |             this._board[castlingTo] ||
673 |             this._attacked(them, this._kings[us]) ||
674 |             this._attacked(them, castlingFrom + 1) ||
675 |             this._attacked(them, castlingTo) ||
676 |             addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.KSIDE_CASTLE);
677 |         }
678 |         if (this._castling[us] & BITS.QSIDE_CASTLE) {
679 |             const castlingFrom = this._kings[us],
680 |                 castlingTo = castlingFrom - 2;
681 |             this._board[castlingFrom - 1] ||
682 |             this._board[castlingFrom - 2] ||
683 |             this._board[castlingFrom - 3] ||
684 |             this._attacked(them, this._kings[us]) ||
685 |             this._attacked(them, castlingFrom - 1) ||
686 |             this._attacked(them, castlingTo) ||
687 |             addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.QSIDE_CASTLE);
688 |         }
689 |     }
690 |     if (!legal || -1 === this._kings[us]) return moves;
691 |     const legalMoves = [];
692 |     for (let i = 0, len = moves.length; i < len; i++)
693 |         (this._makeMove(moves[i]),
694 |         this._isKingAttacked(us) || legalMoves.push(moves[i]),
695 |         this._undoMove());
696 |     return legalMoves;
697 | }
698 | move(move) {

```

```

699 |         let moveObj = null;
700 |         const moves = this._moves();
701 |         for (let i = 0, len = moves.length; i < len; i++)
702 |         {
703 |             if (
704 |                 move.from === algebraic(moves[i].from) &&
705 |                 move.to === algebraic(moves[i].to) &&
706 |                 (!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
707 |             ) {
708 |                 moveObj = moves[i];
709 |                 break;
710 |             }
711 |             if (!moveObj) throw new Error(`Invalid move: ${JSON.stringify(move)}`);
712 |             const prettyMove = new Move(this, moveObj);
713 |             return (this._makeMove(moveObj), this._incPositionCount(), prettyMove);
714 |         }
715 |         _push(move) {
716 |             this._history.push({
717 |                 move: move,
718 |                 kings: {b: this._kings.b, w: this._kings.w},
719 |                 turn: this._turn,
720 |                 castling: {b: this._castling.b, w: this._castling.w},
721 |                 epSquare: this._epSquare,
722 |                 halfMoves: this._halfMoves,
723 |                 moveNumber: this._moveNumber,
724 |             });
725 |         }
726 |         _movePiece(from, to) {
727 |             ((this._hash ^= this._pieceKey(from)),
728 |             (this._board[to] = this._board[from]),
729 |             delete this._board[from],
730 |             (this._hash ^= this._pieceKey(to)));
731 |         }
732 |         _makeMove(move) {
733 |             const us = this._turn,
734 |                   them = swapColor(us);
735 |             if ((this._push(move), move.flags & BITS.NULL_MOVE))
736 |                 return (
737 |                     'b' === us && this._moveNumber++,
738 |                     this._halfMoves++,
739 |                     (this._turn = them),
740 |                     void (this._epSquare = -1)
741 |                 );
742 |             if (
743 |                 ((this._hash ^= this._epKey()),
744 |                 (this._hash ^= this._castlingKey()),
745 |                 move.captured && (this._hash ^= this._pieceKey(move.to)),
746 |                 this._movePiece(move.from, move.to),
747 |                 move.flags & BITS.EP_CAPTURE &&
748 |                 ('b' === this._turn ? this._clear(move.to - 16) : this._clear(move.to + 16)),
749 |                 move.promotion &&
750 |                 (this._clear(move.to), this._set(move.to, {type: move.promotion, color: us})),
751 |                 'k' === this._board[move.to].type
752 |             ) {
753 |                 if (((this._kings[us] = move.to), move.flags & BITS.KSIDE_CASTLE)) {
754 |                     const castlingTo = move.to - 1,
755 |                           castlingFrom = move.to + 1;
756 |                     this._movePiece(castlingFrom, castlingTo);
757 |                 } else if (move.flags & BITS.QSIDE_CASTLE) {
758 |                     const castlingTo = move.to + 1,
759 |                           castlingFrom = move.to - 2;
760 |                     this._movePiece(castlingFrom, castlingTo);
761 |                 }
762 |                 this._castling[us] = 0;
763 |             }
764 |             if (this._castling[us])
765 |                 for (let i = 0, len = ROOKS[us].length; i < len; i++)
766 |                     if (move.from === ROOKS[us][i].square && this._castling[us] & ROOKS[us][i].flag) {
767 |                         this._castling[us] ^= ROOKS[us][i].flag;
768 |                         break;
769 |                     }
770 |             if (this._castling[them])

```

```

770 |         for (let i = 0, len = ROOKS[them].length; i < len; i++)
771 |         {
772 |             if (move.to === ROOKS[them][i].square && this._castling[them] & ROOKS[them][i].flag) {
773 |                 this._castling[them] ^= ROOKS[them][i].flag;
774 |                 break;
775 |             }
776 |         }
777 |         if (((this._hash ^= this._castlingKey()), move.flags & BITS.BIG_PAWN)) {
778 |             let epSquare;
779 |             ((epSquare = 'b' === us ? move.to - 16 : move.to + 16),
780 |              ((move.to - 1) & 136) ||
781 |              ('p' !== this._board[move.to - 1] ? type : ||
782 |              this._board[move.to - 1] ? color : them) &&
783 |              ((move.to + 1) & 136) ||
784 |              ('p' !== this._board[move.to + 1] ? type : ||
785 |              this._board[move.to + 1] ? color : them)
786 |              ? (this._epSquare = -1)
787 |              : ((this._epSquare = epSquare), (this._hash ^= this._epKey())));
788 |             else this._epSquare = -1;
789 |             ('p' === move.piece || move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)
790 |              ? (this._halfMoves = 0)
791 |              : this._halfMoves++,
792 |              'b' === us && this._moveNumber++,
793 |              (this._turn = them),
794 |              (this._hash ^= SIDE_KEY));
795 |         }
796 |         undo() {
797 |             const hash = this._hash,
798 |                   move = this._undoMove();
799 |             if (move) {
800 |                 const prettyMove = new Move(this, move);
801 |                 return (this._decPositionCount(hash), prettyMove);
802 |             }
803 |             return null;
804 |         }
805 |         _undoMove() {
806 |             const old = this._history.pop();
807 |             if (void 0 === old) return null;
808 |             ((this._hash ^= this._epKey()), (this._hash ^= this._castlingKey()));
809 |             const move = old.move;
810 |             ((this._kings = old.kings),
811 |              (this._turn = old.turn),
812 |              (this._castling = old.castling),
813 |              (this._epSquare = old.epSquare),
814 |              (this._halfMoves = old.halfMoves),
815 |              (this._moveNumber = old.moveNumber),
816 |              (this._hash ^= this._epKey()),
817 |              (this._hash ^= this._castlingKey()),
818 |              (this._hash ^= SIDE_KEY));
819 |             const us = this._turn,
820 |                   them = swapColor(us);
821 |             if (move.flags & BITS.NULL_MOVE) return move;
822 |             if (
823 |                 (this._movePiece(move.to, move.from),
824 |                 move.piece &&
825 |                 (this._clear(move.from), this._set(move.from, { type: move.piece, color: us })),
826 |                 move.captured)
827 |             )
828 |             {
829 |                 if (move.flags & BITS.EP_CAPTURE) {
830 |                     let index;
831 |                     ((index = 'b' === us ? move.to - 16 : move.to + 16),
832 |                      this._set(index, { type: 'p', color: them }));
833 |                 } else this._set(move.to, { type: move.captured, color: them });
834 |             }
835 |             if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
836 |                 let castlingTo, castlingFrom;
837 |                 (move.flags & BITS.KSIDE_CASTLE
838 |                  ? ((castlingTo = move.to + 1), (castlingFrom = move.to - 1))
839 |                  : ((castlingTo = move.to - 2), (castlingFrom = move.to + 1)),
840 |                  this._movePiece(castlingFrom, castlingTo));
841 |             }
842 |             return move;
843 |         }
844 |     }
845 |     _moveToSan(move, moves) {

```

```

841 |     let output = '';
842 |     if (move.flags & BITS.KSIDE_CASTLE) output = '0-0';
843 |     else if (move.flags & BITS.QSIDE_CASTLE) output = '0-0-0';
844 |     else {
845 |         if ('p' !== move.piece) {
846 |             const disambiguator = (function getDisambiguator(move, moves) {
847 |                 const from = move.from,
848 |                     to = move.to,
849 |                     piece = move.piece;
850 |                 let ambiguities = 0,
851 |                     sameRank = 0,
852 |                     sameFile = 0;
853 |                 for (let i = 0, len = moves.length; i < len; i++) {
854 |                     const ambigFrom = moves[i].from,
855 |                         ambigTo = moves[i].to;
856 |                     piece === moves[i].piece &&
857 |                         from !== ambigFrom &&
858 |                         to === ambigTo &&
859 |                         (ambiguities++,
860 |                          rank(from) === rank(ambigFrom) && sameRank++,
861 |                          file(from) === file(ambigFrom) && sameFile++);
862 |                 }
863 |                 return ambiguities > 0
864 |                     ? sameRank > 0 && sameFile > 0
865 |                       ? algebraic(from)
866 |                         : sameFile > 0
867 |                           ? algebraic(from).charAt(1)
868 |                             : algebraic(from).charAt(0)
869 |                               : '';
870 |             })(move, moves);
871 |             output += move.piece.toUpperCase() + disambiguator;
872 |         }
873 |         (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE) &&
874 |          ('p' === move.piece && (output += algebraic(move.from)[0]), (output += 'x')),
875 |          (output += algebraic(move.to)),
876 |          move.promotion && (output += '=' + move.promotion.toUpperCase()));
877 |     }
878 |     return (
879 |         this._makeMove(move),
880 |         this.isCheck() && (this.isCheckmate() ? (output += '#') : (output += '+')),
881 |         this._undoMove(),
882 |         output
883 |     );
884 | }
885 | turn() {
886 |     return this._turn;
887 | }
888 | board() {
889 |     const output = [];
890 |     let row = [];
891 |     for (let i = 0x88.a8; i ≤ 0x88.h1; i++)
892 |         (null === this._board[i]
893 |          ? row.push(null)
894 |          : row.push({
895 |              square: algebraic(i),
896 |              type: this._board[i].type,
897 |              color: this._board[i].color,
898 |          })),
899 |         (i + 1) & 136 && (output.push(row), (row = []), (i += 8));
900 |     return output;
901 | }
902 | squareColor(square) {
903 |     if (square in 0x88) {
904 |         const sq = 0x88[square];
905 |         return (rank(sq) + file(sq)) % 2 === 0 ? 'light' : 'dark';
906 |     }
907 |     return null;
908 | }
909 | history({ verbose: verbose = !1 } = {}) {
910 |     const reversedHistory = [],
911 |         moveHistory = [];

```

```

912 |     for (; this._history.length > 0;) reversedHistory.push(this._undoMove());
913 |     for (;;) {
914 |         const move = reversedHistory.pop();
915 |         if (!move) break;
916 |         (verbose
917 |         ? moveHistory.push(new Move(this, move))
918 |         : moveHistory.push(this._moveToSan(move, this._moves()))),
919 |         this._makeMove(move));
920 |     }
921 |     return moveHistory;
922 | }
923 | _getPositionCount(hash) {
924 |     return this._positionCount.get(hash) ?? 0;
925 | }
926 | _incPositionCount() {
927 |     this._positionCount.set(this._hash, (this._positionCount.get(this._hash) ?? 0) + 1);
928 | }
929 | _decPositionCount(hash) {
930 |     const currentCount = this._positionCount.get(hash) ?? 0;
931 |     1 === currentCount
932 |     ? this._positionCount.delete(hash)
933 |     : this._positionCount.set(hash, currentCount - 1);
934 | }
935 | moveNumber() {
936 |     return this._moveNumber;
937 | }
938 | }
939 | const PIECE_VALUES = { p: 100, n: 320, b: 330, r: 500, q: 900, k: 0 };
940 | function searchRoot(chess, depth, ctx, previousBestMove) {
941 |     let alpha = -1e7;
942 |     const candidates = [],
943 |         legalMoves = orderMoves(chess.moves({ verbose: !0 }), previousBestMove);
944 |     for (const move of legalMoves) {
945 |         (checkTime(ctx), chess.move(toMoveCommand(move)));
946 |         const score = -negamax(chess, depth - 1, -1e7, -alpha, 1, ctx);
947 |         (chess.undo(),
948 |         candidates.push({ move: toMoveCommand(move), score: score })),
949 |         (alpha = Math.max(alpha, score)));
950 |     }
951 |     if (0 === candidates.length) throw new SearchTimeout();
952 |     const principal = (function selectBestRootMove(candidates) {
953 |         return candidates.reduce((currentBest, candidate) =>
954 |             candidate.score > currentBest.score ? candidate : currentBest,
955 |             {});
956 |     })(candidates);
957 |     return { selected: selectRootMove(candidates, ctx.settings, principal), principal: principal };
958 | }
959 | function negamax(chess, depth, alpha, beta, ply, ctx) {
960 |     if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
961 |     if (depth <= 0)
962 |         return ctx.settings.quiescence
963 |         ? quiescence(chess, alpha, beta, ply, ctx)
964 |         : evaluatePosition(chess);
965 |     let bestScore = -1e7,
966 |         currentAlpha = alpha;
967 |     const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null);
968 |     for (const move of legalMoves) {
969 |         chess.move(toMoveCommand(move));
970 |         const score = -negamax(chess, depth - 1, -beta, -currentAlpha, ply + 1, ctx);
971 |         if (
972 |             (chess.undo(),
973 |             (bestScore = Math.max(bestScore, score)),
974 |             (currentAlpha = Math.max(currentAlpha, score)),
975 |             currentAlpha >= beta)
976 |         )
977 |             break;
978 |     }
979 |     return bestScore;
980 | }
981 | function quiescence(chess, alpha, beta, ply, ctx) {
982 |     if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);

```

```

983 |   let currentAlpha = -alpha;
984 |   const standPat = evaluatePosition(chess);
985 |   if (standPat ≥ beta) return beta;
986 |   if (((currentAlpha = Math.max(currentAlpha, standPat)), ply ≥ 6)) return currentAlpha;
987 |   const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null),
988 |   tacticalMoves = chess.isCheck() ? legalMoves : legalMoves.filter(isTacticalMove);
989 |   for (const move of tacticalMoves) {
990 |     chess.move(toMoveCommand(move));
991 |     const score = -quiescence(chess, -beta, -currentAlpha, ply + 1, ctx);
992 |     if ((chess.undo(), score ≥ beta)) return beta;
993 |     currentAlpha = Math.max(currentAlpha, score);
994 |   }
995 |   return currentAlpha;
996 | }
997 | function evaluatePosition(chess) {
998 |   const board = chess.board(),
999 |   pawns = [],
1000 |   bishops = { w: 0, b: 0 },
1001 |   kings = {};
1002 |   let whiteScore = 0,
1003 |   blackScore = 0,
1004 |   nonPawnMaterial = 0;
1005 |   for (const row of board)
1006 |     for (const pieceOnSquare of row) {
1007 |       if (null === pieceOnSquare) continue;
1008 |       const { color, type, square } = pieceOnSquare,
1009 |       pieceScore = PIECE_VALUES[type] + pieceSquareScore(type, color, square);
1010 |       ('w' === color ? (whiteScore += pieceScore) : (blackScore += pieceScore),
1011 |       'p' === type
1012 |       ? pawns.push({ color, file: fileIndex(square), rank: rankNumber(square) })
1013 |       : ('k' === type && (nonPawnMaterial += PIECE_VALUES[type]),
1014 |       'b' === type && (bishops[color] += 1),
1015 |       'k' === type && (kings[color] = square)));
1016 |     }
1017 |   ((whiteScore += bishops.w ≥ 2 ? 35 : 0),
1018 |   (blackScore += bishops.b ≥ 2 ? 35 : 0),
1019 |   (whiteScore += pawnStructureScore(pawns, 'w')),
1020 |   (blackScore += pawnStructureScore(pawns, 'b')),
1021 |   (whiteScore += developmentScore(board, 'w')),
1022 |   (blackScore += developmentScore(board, 'b')),
1023 |   (whiteScore += kingSafetyScore(kings.w, pawns, 'w', nonPawnMaterial)),
1024 |   (blackScore += kingSafetyScore(kings.b, pawns, 'b', nonPawnMaterial)));
1025 |   let score = (whiteScore - blackScore) * ('w' === chess.turn() ? 1 : -1);
1026 |   return ((score += 2 * chess.moves().length), chess.isCheck() && (score -= 45), score);
1027 | }
1028 | function pieceSquareScore(type, color, square) {
1029 |   const file2 = fileIndex(square),
1030 |   relativeRank = 'w' === color ? rankNumber(square) : 9 - rankNumber(square),
1031 |   centerDistance = Math.abs(file2 - 3.5) + Math.abs(rankNumber(square) - 4.5);
1032 |   switch (type) {
1033 |     case 'p':
1034 |       return 7 * relativeRank + (file2 ≥ 2 && file2 ≤ 5 ? 8 : 0);
1035 |     case 'n':
1036 |       return 42 - 12 * centerDistance - (1 === relativeRank ? 12 : 0);
1037 |     case 'b':
1038 |       return 30 - 7 * centerDistance;
1039 |     case 'r':
1040 |       return (relativeRank ≥ 7 ? 18 : 0) + (0 === file2 || 7 === file2 ? 4 : 0);
1041 |     case 'q':
1042 |       return 14 - 3 * centerDistance;
1043 |     case 'k':
1044 |       return relativeRank ≥ 7 ? -18 : 0;
1045 |   }
1046 | }
1047 | function pawnStructureScore(pawns, color) {
1048 |   const ownPawns = pawns.filter((pawn) => pawn.color === color),
1049 |   fileCounts = Array.from({ length: 8 }, () => 0);
1050 |   let score = 0;
1051 |   for (const pawn of ownPawns) fileCounts[pawn.file] += 1;
1052 |   for (const pawn of ownPawns) {
1053 |     const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;

```

```

1054 | | | | (fileCounts[pawn.file] > 1 && (score -= 14),
1055 | | | | 0 === (fileCounts[pawn.file - 1] ?? 0) &&
1056 | | | | 0 === (fileCounts[pawn.file + 1] ?? 0) &&
1057 | | | | (score -= 10),
1058 | | | | isPassedPawn(pawn, pawns) && (score += (relativeRank - 1) * (relativeRank - 1) * 9));
1059 | | | }
1060 | | | return score;
1061 | | }
1062 | | function isPassedPawn(pawn, pawns) {
1063 | |   const enemyColor = (function oppositeColor(color) {
1064 | |     return 'w' === color ? 'b' : 'w';
1065 | |   })(pawn.color);
1066 | |   return !pawns.some(
1067 | |     (candidate) =>
1068 | |     !(candidate.color !== enemyColor || Math.abs(candidate.file - pawn.file) > 1) &&
1069 | |     ('w' === pawn.color ? candidate.rank > pawn.rank : candidate.rank < pawn.rank),
1070 | |   );
1071 | | }
1072 | | function developmentScore(board, color) {
1073 | |   const homeRank = 'w' === color ? '1' : '8',
1074 | |     knightHomeSquares = 'w' === color ? new Set(['b1', 'g1']) : new Set(['b8', 'g8']),
1075 | |     bishopHomeSquares = 'w' === color ? new Set(['c1', 'f1']) : new Set(['c8', 'f8']);
1076 | |   let score = 0;
1077 | |   for (const row of board)
1078 | |     for (const piece of row)
1079 | |       if (null !== piece &&
1080 | |         piece.color === color &&
1081 | |         ('n' !== piece.type || knightHomeSquares.has(piece.square) || (score += 18),
1082 | |         'b' !== piece.type || bishopHomeSquares.has(piece.square) || (score += 16),
1083 | |         'q' === piece.type &&
1084 | |         piece.square.endsWith(homeRank) &&
1085 | |         hasMovedMinorPieces(board, color) &&
1086 | |         (score -= 12)));
1087 | |   return score;
1088 | | }
1089 | | function hasMovedMinorPieces(board, color) {
1090 | |   const homeSquares =
1091 | |     'w' === color ? new Set(['b1', 'c1', 'f1', 'g1']) : new Set(['b8', 'c8', 'f8', 'g8']);
1092 | |   for (const row of board)
1093 | |     for (const piece of row)
1094 | |       if (
1095 | |         null !== piece &&
1096 | |         piece.color === color &&
1097 | |         ('n' === piece.type || 'b' === piece.type) &&
1098 | |         homeSquares.has(piece.square)
1099 | |       )
1100 | |         return !1;
1101 | |   return !0;
1102 | | }
1103 | | function kingSafetyScore(kingSquare, pawns, color, nonPawnMaterial) {
1104 | |   if (!kingSquare || nonPawnMaterial < 1800) return 0;
1105 | |   const kingFile = fileIndex(kingSquare),
1106 | |     kingRank = rankNumber(kingSquare),
1107 | |     shieldRank = 'w' === color ? kingRank + 1 : kingRank - 1;
1108 | |   let score = 2 === kingFile || 6 === kingFile ? 28 : -18;
1109 | |   for (let file2 = kingFile - 1; file2 <= kingFile + 1; file2 += 1)
1110 | |     if (file2 < 0 ||
1111 | |       file2 >= 8 ||
1112 | |       (score += pawns.some(
1113 | |         (pawn) => pawn.color === color && pawn.file === file2 && pawn.rank === shieldRank,
1114 | |       )
1115 | |       ? 14
1116 | |       : -10));
1117 | |   return score;
1118 | | }
1119 | | function terminalScore(chess, ply) {
1120 | |   return chess.isCheckmate() ? -1e6 + ply : chess.isDraw() ? 0 : evaluatePosition(chess);
1121 | | }
1122 | | function orderMoves(moves, preferredMove) {
1123 | |   return [...moves].sort(
1124 | |     (left, right) =>

```



```

1125 |     moveOrderingScore(right, preferredMove) -- moveOrderingScore(left, preferredMove),
1126 | );
1127 | }
1128 | function moveOrderingScore(move, preferredMove) {
1129 |     let score = 0;
1130 |     return (
1131 |         preferredMove &&
1132 |         (function isSameMove(move, command) {
1133 |             return (
1134 |                 move.from === command.from &&
1135 |                 move.to === command.to &&
1136 |                 (move.promotion ?? void 0) === (command.promotion ?? void 0)
1137 |             );
1138 |         })(move, preferredMove) &&
1139 |         (score += 2e6),
1140 |         move.san.includes('#') ? (score += 15e5) : move.san.includes('+') && (score += 16e3),
1141 |         move.isCapture() &&
1142 |         (score +=
1143 |             12 * (move.captured ? PIECE_VALUES[move.captured] : PIECE_VALUES.p) --
1144 |             PIECE_VALUES[move.piece]),
1145 |         move.isPromotion() && move.promotion && (score += PIECE_VALUES[move.promotion] + 8e3),
1146 |         (move.isKingsideCastle() || move.isQueensideCastle()) && (score += 1500),
1147 |         (score += 2 * pieceSquareScore(move.piece, move.color, move.to)),
1148 |         score
1149 |     );
1150 | }
1151 | function checkTime(ctx) {
1152 |     if (((ctx.nodes += 1), ctx.nodes % 2048 === 0 && Date.now() >= ctx.deadline))
1153 |         throw new SearchTimeout();
1154 | }
1155 | function isTacticalMove(move) {
1156 |     return (
1157 |         move.isCapture() || move.isPromotion() || move.san.includes('+') || move.san.includes('#')
1158 |     );
1159 | }
1160 | function selectRootMove(candidates, settings, best) {
1161 |     if (
1162 |         0 === settings.randomness ||
1163 |         (function isMateProtectedScore(score) {
1164 |             return Math.abs(score) >= 99e4;
1165 |         })(best.score)
1166 |     )
1167 |         return best;
1168 |     const level = settings.randomness / 30,
1169 |           windowCp = 20 + 150 * level,
1170 |           floorScore = best.score - windowCp,
1171 |           ticketPower = 4 - 1.5 * level,
1172 |           weightedCandidates = candidates
1173 |             .map((candidate) => ({
1174 |                 candidate: candidate,
1175 |                 tickets: (Math.max(0, candidate.score - floorScore) / windowCp) ** ticketPower,
1176 |             }));
1177 |     .filter(({ tickets: tickets }) => tickets > 0),
1178 |     totalTickets = weightedCandidates.reduce((total, { tickets: tickets }) => total + tickets, 0);
1179 |     if (totalTickets <= 0) return best;
1180 |     let draw = Math.random() * totalTickets;
1181 |     for (const { candidate: candidate, tickets: tickets } of weightedCandidates)
1182 |         if (((draw -= tickets), draw <= 0)) return candidate;
1183 |     return best;
1184 | }
1185 | function toMoveCommand(move) {
1186 |     return move.promotion
1187 |       ? { from: move.from, to: move.to, promotion: move.promotion }
1188 |       : { from: move.from, to: move.to };
1189 | }
1190 | function fileIndex(square) {
1191 |     return square.charCodeAt(0) - 97;
1192 | }
1193 | function rankNumber(square) {
1194 |     return Number(square[1]);
1195 | }

```

```

1196 | function clamp(value, min, max) {
1197 |     return Math.min(max, Math.max(min, value));
1198 | }
1199 | class SearchTimeout extends Error {}
1200 | const workerScope = globalThis;
1201 | workerScope.addEventListener('message', (event) => {
1202 |     const response = (function searchBestMove(request, onProgress) {
1203 |         const chess = new Chess(request.fen),
1204 |             settings = (function normalizeAiSettings(settings) {
1205 |                 return {
1206 |                     preset: settings.preset,
1207 |                     maxDepth: ((depth = settings.maxDepth), clamp(Math.round(depth), 1, 5)),
1208 |                     timeLimitMs: clamp(Math.round(settings.timeLimitMs), 500, 5e3),
1209 |                     randomness: clamp(Math.round(settings.randomness), 0, 30),
1210 |                     quiescence: settings.quiescence,
1211 |                 };
1212 |                 var depth;
1213 |             })(request.settings),
1214 |             legalMoves = chess.moves({ verbose: !0 }),
1215 |             startTime = Date.now(),
1216 |             ctx = {
1217 |                 settings: settings,
1218 |                 deadline: startTime + settings.timeLimitMs,
1219 |                 startTime: startTime,
1220 |                 nodes: 0,
1221 |             };
1222 |             if (0 === legalMoves.length)
1223 |                 return {
1224 |                     id: request.id,
1225 |                     move: null,
1226 |                     depthReached: 0,
1227 |                     nodes: 0,
1228 |                     elapsedMs: 0,
1229 |                     score: terminalScore(chess, 0),
1230 |                 };
1231 |             let best = null,
1232 |                 previousBestMove = null,
1233 |                 depthReached = 0;
1234 |             for (let depth = 1; depth ≤ settings.maxDepth; depth += 1) {
1235 |                 try {
1236 |                     const result = searchRoot(chess, depth, ctx, previousBestMove);
1237 |                     ((best = result.selected),
1238 |                     (previousBestMove = result.principal.move),
1239 |                     (depthReached = depth),
1240 |                     onProgress?.({
1241 |                         kind: 'progress',
1242 |                         id: request.id,
1243 |                         move: best.move,
1244 |                         depthReached: depthReached,
1245 |                         nodes: ctx.nodes,
1246 |                         elapsedMs: Date.now() - startTime,
1247 |                         score: best.score,
1248 |                     }));
1249 |                 } catch (error) {
1250 |                     if (error instanceof SearchTimeout) break;
1251 |                     throw error;
1252 |                 }
1253 |                 if (Date.now() ≥ ctx.deadline) break;
1254 |             }
1255 |             return (
1256 |             (best ??= (function chooseFallbackMove(chess) {
1257 |                 const [move] = orderMoves(chess.moves({ verbose: !0 }), null);
1258 |                 if (!move) return { move: { from: 'a1', to: 'a1' }, score: terminalScore(chess, 0) };
1259 |                 chess.move(toMoveCommand(move));
1260 |                 const score = evaluatePosition(chess);
1261 |                 return (chess.undo(), { move: toMoveCommand(move), score: score });
1262 |             })(chess)),
1263 |             {
1264 |                 id: request.id,
1265 |                 move: best.move,
1266 |                 depthReached: depthReached,

```

```

1267 |         nodes: ctx.nodes,
1268 |         elapsedMs: Date.now() - startTime,
1269 |         score: best.score,
1270 |     }
1271 | );
1272 | })(event.data, (progress) => {
1273 |     workerScope.postMessage(progress);
1274 | });
1275 | workerScope.postMessage({ ... response, kind: 'result' });
1276 | });
1277 | })();
1278 |

```

==== assets/index-BVkn0d8S.css (10778 B / 969ee06a92c7) ====

```

1 | *,
2 | *:before,
3 | *:after {
4 |     box-sizing: border-box;
5 | }
6 | html {
7 |     min-width: 320px;
8 |     color-scheme: light;
9 | }
10 | body {
11 |     min-width: 320px;
12 |     min-height: 100vh;
13 |     margin: 0;
14 |     font-family:
15 |         Inter,
16 |         ui-sans-serif,
17 |         system-ui,
18 |         -apple-system,
19 |         BlinkMacSystemFont,
20 |         Segoe UI,
21 |         sans-serif;
22 |     text-rendering: geometricPrecision;
23 | }
24 | button {
25 |     font: inherit;
26 | }
27 | button:not(:disabled) {
28 |     cursor: pointer;
29 | }
30 | button:disabled {
31 |     cursor: not-allowed;
32 | }
33 | :root {
34 |     --app-padding: clamp(12px, 2.5vw, 28px);
35 |     --page: #f3eee3;
36 |     --ink: #20252b;
37 |     --muted: #66716f;
38 |     --line: #c9c0b1;
39 |     --panel: #fffaf0;
40 |     --button: #263a3c;
41 |     --button-text: #ffffff;
42 |     --accent: #b94b36;
43 |     --light-square: #eadbb7;
44 |     --dark-square: #5f8a65;
45 |     --last-move: #e0b84b;
46 |     --legal: #264f8f;
47 |     --capture: #b83232;
48 |     --shadow: 0 18px 42px rgb(36 31 24 / 18%);
49 | }
50 | .chess-app {
51 |     --board-max-size: min(68dvh, 620px);
52 |     height: 100dvh;
53 |     display: grid;
54 |     grid-template-columns: minmax(210px, 260px) minmax(320px, var(--board-max-size)) minmax(
55 |         104px,
56 |         132px
57 |     ) minmax(240px, 310px);

```

```

58 | gap: clamp(10px, 1.4vw, 20px);
59 | align-items: stretch;
60 | justify-content: center;
61 | overflow: hidden;
62 | padding: var(--app-padding);
63 | background: var(--page);
64 | color: var(--ink);
65 | }
66 | .board-stage {
67 |   width: min(100%, var(--board-max-size), 100dvh - var(--app-padding) * 2);
68 |   align-self: center;
69 |   justify-self: center;
70 | }
71 | .clock-panel {
72 |   width: 100%;
73 |   height: min(var(--board-max-size), 100dvh - var(--app-padding) * 2);
74 |   min-width: 0;
75 |   min-height: 0;
76 |   align-self: center;
77 |   justify-self: stretch;
78 |   display: flex;
79 | }
80 | .chess-board {
81 |   width: 100%;
82 |   aspect-ratio: 1;
83 |   display: grid;
84 |   grid-template-columns: repeat(8, minmax(0, 1fr));
85 |   overflow: hidden;
86 |   border: 3px solid #2e3a32;
87 |   border-radius: 8px;
88 |   box-shadow: var(--shadow);
89 |   background: #2e3a32;
90 | }
91 | .board-square {
92 |   position: relative;
93 |   width: 100%;
94 |   aspect-ratio: 1;
95 |   display: grid;
96 |   place-items: center;
97 |   min-width: 0;
98 |   padding: 0;
99 |   border: 0;
100 |   color: var(--ink);
101 |   transition:
102 |     box-shadow 0.14s ease,
103 |     filter 0.14s ease,
104 |     transform 0.14s ease;
105 | }
106 | .board-square:not(:disabled):hover {
107 |   filter: brightness(1.05);
108 | }
109 | .board-square.is-light {
110 |   background: var(--light-square);
111 | }
112 | .board-square.is-dark {
113 |   background: var(--dark-square);
114 | }
115 | .board-square.is-selected {
116 |   box-shadow: inset 0 0 4px var(--legal);
117 | }
118 | .board-square.is-last-move {
119 |   box-shadow: inset 0 0 4px #e0b84bc2;
120 | }
121 | .board-square.is-selected.is-last-move {
122 |   box-shadow:
123 |     inset 0 0 4px var(--legal),
124 |     inset 0 0 8px #e0b84bad;
125 | }
126 | .board-square.is-legal:after,
127 | .board-square.is-capture:after {
128 |   position: absolute;

```

```

129 |   z-index: 1;
130 |   content: '';
131 |   border-radius: 999px;
132 |   pointer-events: none;
133 | }
134 | .board-square.is-legal:after {
135 |   width: 28%;
136 |   height: 28%;
137 |   background: #264f8f6b;
138 | }
139 | .board-square.is-capture:after {
140 |   inset: 12%;
141 |   border: max(3px, 0.45vmin) solid rgba(184, 50, 50, 0.74);
142 | }
143 | .board-square.is-check {
144 |   box-shadow: inset 0 0 0.5px var(--capture);
145 | }
146 | .chess-board.is-locked .board-square {
147 |   filter: saturate(0.86);
148 | }
149 | .piece {
150 |   z-index: 2;
151 |   display: block;
152 |   font-size: clamp(2rem, 7.8vmin, 5rem);
153 |   line-height: 1;
154 |   text-shadow: 0 2px 6px rgba(0, 0, 0, 0.2);
155 |   -webkit-user-select: none;
156 |   user-select: none;
157 | }
158 | .coordinate {
159 |   position: absolute;
160 |   z-index: 3;
161 |   font-size: clamp(0.58rem, 1.4vmin, 0.76rem);
162 |   font-weight: 800;
163 |   line-height: 1;
164 |   opacity: 0.72;
165 |   pointer-events: none;
166 | }
167 | .coordinate-rank {
168 |   top: 6px;
169 |   left: 7px;
170 | }
171 | .coordinate-file {
172 |   right: 7px;
173 |   bottom: 6px;
174 | }
175 | .control-panel,
176 | .info-panel {
177 |   min-width: 0;
178 |   min-height: 0;
179 |   max-height: calc(100dvh - var(--app-padding) * 2);
180 |   display: flex;
181 |   flex-direction: column;
182 |   gap: 14px;
183 |   overflow: auto;
184 |   overscroll-behavior: contain;
185 |   scrollbar-gutter: stable;
186 | }
187 | .control-panel {
188 |   justify-self: end;
189 | }
190 | .info-panel {
191 |   justify-self: start;
192 | }
193 | .app-header {
194 |   display: grid;
195 |   gap: 4px;
196 | }
197 | .eyebrow,
198 | .control-label {
199 |   margin: 0;

```

```

200 | color: var(--muted);
201 | font-size: 0.78rem;
202 | font-weight: 800;
203 | letter-spacing: 0;
204 | text-transform: uppercase;
205 | }
206 | .app-header h1 {
207 | margin: 0;
208 | font-size: clamp(2rem, 4vw, 3rem);
209 | line-height: 1;
210 | letter-spacing: 0;
211 | }
212 | .control-group,
213 | .status-panel,
214 | .history-panel {
215 | display: grid;
216 | gap: 10px;
217 | padding-top: 16px;
218 | border-top: 1px solid var(--line);
219 | }
220 | .history-panel {
221 | min-height: 0;
222 | flex: 1;
223 | grid-template-rows: auto minmax(0, 1fr);
224 | }
225 | .segmented-control {
226 | display: grid;
227 | grid-template-columns: repeat(2, minmax(0, 1fr));
228 | gap: 4px;
229 | padding: 4px;
230 | border: 1px solid var(--line);
231 | border-radius: 8px;
232 | background: #fffaf0ad;
233 | }
234 | .segmented-control-three {
235 | grid-template-columns: repeat(3, minmax(0, 1fr));
236 | }
237 | .clock-preset-control {
238 | display: grid;
239 | grid-template-columns: repeat(5, minmax(0, 1fr));
240 | gap: 4px;
241 | padding: 4px;
242 | border: 1px solid var(--line);
243 | border-radius: 8px;
244 | background: #fffaf0ad;
245 | }
246 | .segmented-button,
247 | .primary-button,
248 | .ghost-button,
249 | .promotion-button {
250 | min-height: 42px;
251 | border-radius: 6px;
252 | font-weight: 800;
253 | letter-spacing: 0;
254 | }
255 | .segmented-button {
256 | border: 0;
257 | background: transparent;
258 | color: var(--muted);
259 | }
260 | .segmented-button.is-active {
261 | background: var(--button);
262 | color: var(--button-text);
263 | }
264 | .clock-settings .segmented-button {
265 | min-height: 36px;
266 | padding-inline: 4px;
267 | font-size: 0.82rem;
268 | }
269 | .ai-settings {
270 | gap: 12px;

```

```

271 | }
272 | .range-control,
273 | .toggle-control-{
274 |   display: grid;
275 |   gap: 8px;
276 |   min-width: 0;
277 |   color: var(--ink);
278 |   font-size: 0.92rem;
279 |   font-weight: 800;
280 | }
281 | .range-control span,
282 | .toggle-control-{
283 |   align-items: center;
284 | }
285 | .range-control span-{
286 |   display: flex;
287 |   justify-content: space-between;
288 |   gap: 10px;
289 | }
290 | .range-control strong-{
291 |   color: var(--accent);
292 |   white-space: nowrap;
293 | }
294 | .range-control input[type='range']-{
295 |   width: 100%;
296 |   accent-color: var(--button);
297 | }
298 | .toggle-control-{
299 |   grid-template-columns: auto minmax(0, 1fr);
300 |   padding: 10px;
301 |   border: 1px solid var(--line);
302 |   border-radius: 8px;
303 |   background: #fffaf0bd;
304 | }
305 | .toggle-control input-{
306 |   width: 18px;
307 |   height: 18px;
308 |   accent-color: var(--button);
309 | }
310 | .action-row-{
311 |   display: grid;
312 |   grid-template-columns: 1fr 1fr;
313 |   gap: 10px;
314 | }
315 | .primary-button,
316 | .ghost-button-{
317 |   border: 1px solid var(--button);
318 | }
319 | .primary-button-{
320 |   background: var(--button);
321 |   color: var(--button-text);
322 | }
323 | .ghost-button-{
324 |   background: transparent;
325 |   color: var(--button);
326 | }
327 | .primary-button:disabled,
328 | .ghost-button:disabled,
329 | .segmented-button:disabled-{
330 |   opacity: 0.46;
331 | }
332 | .game-status-{
333 |   min-height: 1.4em;
334 |   color: var(--accent);
335 |   font-size: 1.35rem;
336 |   line-height: 1.2;
337 | }
338 | .clock-display-{
339 |   width: 100%;
340 |   min-height: 0;
341 |   min-width: 0;

```

```

342 |   display: grid;
343 |   grid-template-rows: repeat(2, minmax(0, 1fr));
344 |   gap: 10px;
345 | }
346 | .clock-card {
347 |   min-width: 0;
348 |   min-height: 0;
349 |   display: grid;
350 |   place-content: center;
351 |   gap: 4px;
352 |   padding: 12px 8px;
353 |   border: 1px solid var(--line);
354 |   border-radius: 8px;
355 |   background: #fffaf0bd;
356 |   text-align: center;
357 | }
358 | .clock-card.is-active {
359 |   border-color: var(--button);
360 |   background: #fff;
361 |   box-shadow: inset 0 0 0 2px #263a3c2e;
362 | }
363 | .clock-card.is-warning .clock-time {
364 |   color: #a05a00;
365 | }
366 | .clock-card.is-danger .clock-time,
367 | .clock-card.is-timeout .clock-time {
368 |   color: var(--capture);
369 | }
370 | .clock-side {
371 |   color: var(--muted);
372 |   font-size: 0.72rem;
373 |   font-weight: 800;
374 |   line-height: 1;
375 | }
376 | .clock-time {
377 |   font-variant-numeric: tabular-nums;
378 |   color: var(--ink);
379 |   font-size: clamp(1.15rem, 1.8vw, 1.55rem);
380 |   line-height: 1;
381 | }
382 | .status-grid {
383 |   display: grid;
384 |   grid-template-columns: 1fr;
385 |   gap: 8px;
386 |   margin: 0;
387 | }
388 | .status-grid.div {
389 |   min-width: 0;
390 |   padding: 10px;
391 |   border: 1px solid var(--line);
392 |   border-radius: 8px;
393 |   background: #fffaf0bd;
394 | }
395 | .status-grid.dt,
396 | .status-grid.dd {
397 |   margin: 0;
398 | }
399 | .status-grid.dt {
400 |   color: var(--muted);
401 |   font-size: 0.72rem;
402 |   font-weight: 800;
403 | }
404 | .status-grid.dd {
405 |   margin-top: 4px;
406 |   overflow-wrap: anywhere;
407 |   font-size: 0.92rem;
408 |   font-weight: 800;
409 | }
410 | .history-header {
411 |   display: flex;
412 |   align-items: center;

```



```

413 | justify-content: space-between;
414 | }
415 | .move-list {
416 | min-height: 0;
417 | max-height: none;
418 | overflow: auto;
419 | border: 1px solid var(--line);
420 | border-radius: 8px;
421 | background: #fffaf0bd;
422 | }
423 | .move-row {
424 | display: grid;
425 | grid-template-columns: 3rem minmax(0, 1fr) minmax(0, 1fr);
426 | gap: 8px;
427 | padding: 8px 10px;
428 | border-bottom: 1px solid rgba(201, 192, 177, 0.62);
429 | font-weight: 800;
430 | }
431 | .move-row:last-child {
432 | border-bottom: 0;
433 | }
434 | .move-number {
435 | color: var(--muted);
436 | }
437 | .move-san {
438 | min-width: 0;
439 | overflow-wrap: anywhere;
440 | }
441 | .move-empty {
442 | margin: 0;
443 | padding: 14px 12px;
444 | color: var(--muted);
445 | font-weight: 700;
446 | }
447 | .promotion-overlay {
448 | position: fixed;
449 | inset: 0;
450 | z-index: 10;
451 | display: grid;
452 | place-items: center;
453 | padding: 20px;
454 | background: #15191c94;
455 | }
456 | .promotion-overlay[hidden] {
457 | display: none;
458 | }
459 | .promotion-dialog {
460 | width: min(92vw, 360px);
461 | display: grid;
462 | gap: 16px;
463 | padding: 20px;
464 | border: 1px solid var(--line);
465 | border-radius: 8px;
466 | background: var(--panel);
467 | box-shadow: var(--shadow);
468 | }
469 | .promotion-dialog h2 {
470 | margin: 0;
471 | font-size: 1.1rem;
472 | }
473 | .promotion-choices {
474 | display: grid;
475 | grid-template-columns: repeat(4, minmax(0, 1fr));
476 | gap: 8px;
477 | }
478 | .promotion-button {
479 | min-height: 70px;
480 | border: 1px solid var(--button);
481 | background: #fff;
482 | color: var(--ink);
483 | font-size: 2.4rem;

```

```

484 |     line-height: 1;
485 | }
486 | @media (max-width: 1180px) {
487 |     .chess-app {
488 |         --board-max-size: min(62dvh, 560px);
489 |         grid-template-columns: minmax(190px, 230px) minmax(300px, var(--board-max-size)) minmax(
490 |             94px,
491 |             118px
492 |         ) minmax(220px, 280px);
493 |         gap: 10px;
494 |     }
495 |     .segmented-button,
496 |     .primary-button,
497 |     .ghost-button,
498 |     .promotion-button {
499 |         min-height: 38px;
500 |     }
501 |     .app-header h1 {
502 |         font-size: clamp(1.7rem, 3vw, 2.4rem);
503 |     }
504 | }
505 | @media (max-width: 960px) {
506 |     .chess-app {
507 |         --board-max-size: min(100%, 640px);
508 |         min-height: 100dvh;
509 |         height: auto;
510 |         grid-template-columns: minmax(0, 1fr);
511 |         overflow: visible;
512 |     }
513 |     .board-stage,
514 |     .control-panel,
515 |     .clock-panel,
516 |     .info-panel {
517 |         width: min(100%, 640px);
518 |         justify-self: center;
519 |     }
520 |     .control-panel,
521 |     .clock-panel,
522 |     .info-panel {
523 |         max-height: none;
524 |         overflow: visible;
525 |     }
526 |     .board-stage {
527 |         order: 2;
528 |     }
529 |     .control-panel {
530 |         order: 1;
531 |     }
532 |     .clock-panel {
533 |         order: 3;
534 |         height: auto;
535 |     }
536 |     .info-panel {
537 |         order: 4;
538 |     }
539 |     .clock-display {
540 |         grid-template-rows: repeat(2, minmax(82px, auto));
541 |     }
542 |     .status-grid {
543 |         grid-template-columns: 1fr;
544 |     }
545 |     .move-list {
546 |         max-height: min(38vh, 360px);
547 |     }
548 | }
549 | @media (max-width: 520px) {
550 |     .chess-app {
551 |         padding: 12px;
552 |         gap: 14px;
553 |     }
554 |     .app-header h1 {

```

```

555 |     font-size: 2rem;
556 | }
557 | .piece {
558 |     font-size: clamp(1.7rem, 10vw, 3.2rem);
559 | }
560 | .coordinate {
561 |     font-size: 0.56rem;
562 | }
563 | .action-row {
564 |     grid-template-columns: 1fr;
565 | }
566 | }
567 |

```

==== assets/index-CSn2cQoE.js (63328 B / 21f79ae3982c) ====

```

1 | const MASK64 = 0xfffffffffffffn;
2 | function rotl(x, k) {
3 |     return 0xfffffffffffffn & ((x << k) | (x >> (64n - k)));
4 | }
5 | function wrappingMul(x, y) {
6 |     return (x * y) & MASK64;
7 | }
8 | const rand = (function xoroshiro128(state) {
9 |     return function () {
10 |         let s0 = BigInt(state & MASK64),
11 |             s1 = BigInt((state >> 64n) & MASK64);
12 |         const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
13 |         return (
14 |             (s1 ^ s0),
15 |             (s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64),
16 |             (s1 = rotl(s1, 37n)),
17 |             (state = (s1 << 64n) | s0),
18 |             result
19 |         );
20 |     };
21 | })(0xa187eb39cdcaed8f31c4b365b102e01en),
22 | PIECE_KEYS = Array.from({ length: 2 }, () =>
23 |     Array.from({ length: 6 }, () => Array.from({ length: 128 }, () => rand()))),
24 | ),
25 | EP_KEYS = Array.from({ length: 8 }, () => rand()),
26 | CASTLING_KEYS = Array.from({ length: 16 }, () => rand()),
27 | SIDE_KEY = rand(),
28 | DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR·w·KQkq--0·1';
29 | class Move {
30 |     color;
31 |     from;
32 |     to;
33 |     piece;
34 |     captured;
35 |     promotion;
36 |     flags;
37 |     san;
38 |     lan;
39 |     before;
40 |     after;
41 |     constructor(chess, internal) {
42 |         const {
43 |             color: color,
44 |             piece: piece,
45 |             from: from,
46 |             to: to,
47 |             flags: flags,
48 |             captured: captured,
49 |             promotion: promotion,
50 |         } = internal,
51 |             fromAlgebraic = algebraic(from),
52 |             toAlgebraic = algebraic(to);
53 |         ((this.color = color),
54 |         (this.piece = piece),
55 |         (this.from = fromAlgebraic),
56 |         (this.to = toAlgebraic),

```

```

57 | | | | | (this.san = chess._moveToSan(internal, chess._moves({ legal: !0 }))),
58 | | | | | (this.lan = fromAlgebraic + toAlgebraic),
59 | | | | | (this.before = chess.fen()),
60 | | | | | chess._makeMove(internal),
61 | | | | | (this.after = chess.fen()),
62 | | | | | chess._undoMove(),
63 | | | | | (this.flags = '');
64 | | | | | for (const flag in BITS) BITS[flag] & flags && (this.flags += FLAGS[flag]);
65 | | | | | (captured && (this.captured = captured),
66 | | | | | promotion && ((this.promotion = promotion), (this.lan += promotion)));
67 | | | }
68 | | | isCapture() {
69 | | | | | return this.flags.indexOf(FLAGS.CAPTURE) > -1;
70 | | | }
71 | | | isPromotion() {
72 | | | | | return this.flags.indexOf(FLAGS.PROMOTION) > -1;
73 | | | }
74 | | | isEnPassant() {
75 | | | | | return this.flags.indexOf(FLAGS.EP_CAPTURE) > -1;
76 | | | }
77 | | | isKingsideCastle() {
78 | | | | | return this.flags.indexOf(FLAGS.KSIDE_CASTLE) > -1;
79 | | | }
80 | | | isQueensideCastle() {
81 | | | | | return this.flags.indexOf(FLAGS.QSIDE_CASTLE) > -1;
82 | | | }
83 | | | isBigPawn() {
84 | | | | | return this.flags.indexOf(FLAGS.BIG_PAWN) > -1;
85 | | | }
86 | | }
87 | const FLAGS = {
88 | | | NORMAL: 'n',
89 | | | CAPTURE: 'c',
90 | | | BIG_PAWN: 'b',
91 | | | EP_CAPTURE: 'e',
92 | | | PROMOTION: 'p',
93 | | | KSIDE_CASTLE: 'k',
94 | | | QSIDE_CASTLE: 'q',
95 | | | NULL_MOVE: '-',
96 | | },
97 | BITS = {
98 | | | NORMAL: 1,
99 | | | CAPTURE: 2,
100 | | | BIG_PAWN: 4,
101 | | | EP_CAPTURE: 8,
102 | | | PROMOTION: 16,
103 | | | KSIDE_CASTLE: 32,
104 | | | QSIDE_CASTLE: 64,
105 | | | NULL_MOVE: 128,
106 | | },
107 | 0x88 = {
108 | | | a8: 0,
109 | | | b8: 1,
110 | | | c8: 2,
111 | | | d8: 3,
112 | | | e8: 4,
113 | | | f8: 5,
114 | | | g8: 6,
115 | | | h8: 7,
116 | | | a7: 16,
117 | | | b7: 17,
118 | | | c7: 18,
119 | | | d7: 19,
120 | | | e7: 20,
121 | | | f7: 21,
122 | | | g7: 22,
123 | | | h7: 23,
124 | | | a6: 32,
125 | | | b6: 33,
126 | | | c6: 34,
127 | | | d6: 35,

```

```

128 | e6: 36,
129 | f6: 37,
130 | g6: 38,
131 | h6: 39,
132 | a5: 48,
133 | b5: 49,
134 | c5: 50,
135 | d5: 51,
136 | e5: 52,
137 | f5: 53,
138 | g5: 54,
139 | h5: 55,
140 | a4: 64,
141 | b4: 65,
142 | c4: 66,
143 | d4: 67,
144 | e4: 68,
145 | f4: 69,
146 | g4: 70,
147 | h4: 71,
148 | a3: 80,
149 | b3: 81,
150 | c3: 82,
151 | d3: 83,
152 | e3: 84,
153 | f3: 85,
154 | g3: 86,
155 | h3: 87,
156 | a2: 96,
157 | b2: 97,
158 | c2: 98,
159 | d2: 99,
160 | e2: 100,
161 | f2: 101,
162 | g2: 102,
163 | h2: 103,
164 | a1: 112,
165 | b1: 113,
166 | c1: 114,
167 | d1: 115,
168 | e1: 116,
169 | f1: 117,
170 | g1: 118,
171 | h1: 119,
172 | },
173 | PAWN_OFFSETS = { b: [16, 32, 17, 15], w: [-16, -32, -17, -15] },
174 | PIECE_OFFSETS = {
175 |   n: [-18, -33, -31, -14, 18, 33, 31, 14],
176 |   b: [-17, -15, 17, 15],
177 |   r: [-16, 1, 16, -1],
178 |   q: [-17, -16, -15, 1, 17, 16, 15, -1],
179 |   k: [-17, -16, -15, 1, 17, 16, 15, -1],
180 | },
181 | ATTACKS = [
182 |   20, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20,
183 |   0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 20,
184 |   0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20,
185 |   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 24, 24, 24, 24, 24, 24, 56,
186 |   0, 56, 24, 24, 24, 24, 24, 24, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
187 |   0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0,
188 |   0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0,
189 |   0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0,
190 |   20,
191 | ],
192 | RAYS = [
193 |   17, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15,
194 |   0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 15, 0, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0, 0, 15,
195 |   0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 0, 17, 0, 16, 0, 15,
196 |   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, -1,
197 |   -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
198 |   -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0, 0, -17, 0, 0, 0, 0, 0, 0,

```

```

199 |     0, 0, -15, 0, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, 0, -17,
200 |     0, 0, 0, 0, -15, 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, -17, 0, 0, -15, 0, 0, 0, 0, 0, -16, 0, 0,
201 |     0, 0, 0, 0, -17,
202 | ],
203 | PIECE_MASKS = {p: 1, n: 2, b: 4, r: 8, q: 16, k: 32},
204 | PROMOTIONS = ['n', 'b', 'r', 'q'],
205 | ROOKS = {
206 |     w: [
207 |         {square: 0x88.a1, flag: BITS.QSIDE_CASTLE},
208 |         {square: 0x88.h1, flag: BITS.KSIDE_CASTLE},
209 |     ],
210 |     b: [
211 |         {square: 0x88.a8, flag: BITS.QSIDE_CASTLE},
212 |         {square: 0x88.h8, flag: BITS.KSIDE_CASTLE},
213 |     ],
214 | },
215 | SECOND_RANK = {b: 1, w: 6};
216 | function rank(square) {
217 |     return square >> 4;
218 | }
219 | function file(square) {
220 |     return 15 & square;
221 | }
222 | function isDigit(c) {
223 |     return -1 !== '0123456789'.indexOf(c);
224 | }
225 | function algebraic(square) {
226 |     const f = file(square),
227 |           r = rank(square);
228 |     return 'abcdefgh'.substring(f, f + 1) + '87654321'.substring(r, r + 1);
229 | }
230 | function swapColor(color) {
231 |     return 'w' === color ? 'b' : 'w';
232 | }
233 | function addMove(moves, color, from, to, piece, captured = void 0, flags = BITS.NORMAL) {
234 |     const r = rank(to);
235 |     if ('p' !== piece || (7 !== r & 0 !== r))
236 |         moves.push({
237 |             color: color,
238 |             from: from,
239 |             to: to,
240 |             piece: piece,
241 |             captured: captured,
242 |             flags: flags,
243 |         });
244 |     else
245 |         for (let i = 0; i < PROMOTIONS.length; i++) {
246 |             const promotion = PROMOTIONS[i];
247 |             moves.push({
248 |                 color: color,
249 |                 from: from,
250 |                 to: to,
251 |                 piece: piece,
252 |                 captured: captured,
253 |                 promotion: promotion,
254 |                 flags: flags | BITS.PROMOTION,
255 |             });
256 |         }
257 | }
258 | class Chess {
259 |     _board = new Array(128);
260 |     _turn = 'w';
261 |     _kings = {w: -1, b: -1};
262 |     _epSquare = -1;
263 |     _halfMoves = 0;
264 |     _moveNumber = 0;
265 |     _history = [];
266 |     _castling = {w: 0, b: 0};
267 |     _hash = 0n;
268 |     _positionCount = new Map();
269 |     constructor(fen = DEFAULT_POSITION, {skipValidation: skipValidation = !1} = {}) {

```

```

270 |     this.load(fen, { skipValidation: skipValidation });
271 | }
272 | clear() {
273 |     ((this._board = new Array(128)),
274 |     (this._kings = { w: -1, b: -1 }),
275 |     (this._turn = 'w'),
276 |     (this._castling = { w: 0, b: 0 }),
277 |     (this._epSquare = -1),
278 |     (this._halfMoves = 0),
279 |     (this._moveNumber = 1),
280 |     (this._history = []),
281 |     (this._hash = this._computeHash()),
282 |     (this._positionCount = new Map()));
283 | }
284 | load(fen, { skipValidation: skipValidation = !1 } = {}) {
285 |     let tokens = fen.split(/\s+/);
286 |     if (tokens.length ≥ 2 && tokens.length < 6) {
287 |         const adjustments = ['- ', '- ', '0', '1'];
288 |         fen = tokens.concat(adjustments.slice(-(6 - tokens.length))).join(' ');
289 |     }
290 |     if (((tokens = fen.split(/\s+/)), !skipValidation)) {
291 |         const { ok: ok, error: error } = (function validateFen(fen) {
292 |             const tokens = fen.split(/\s+/);
293 |             if (6 !== tokens.length)
294 |                 return { ok: !1, error: 'Invalid FEN: must contain six space-delimited fields' };
295 |             const moveNumber = parseInt(tokens[5], 10);
296 |             if (isNaN(moveNumber) || moveNumber ≤ 0)
297 |                 return { ok: !1, error: 'Invalid FEN: move number must be a positive integer' };
298 |             const halfMoves = parseInt(tokens[4], 10);
299 |             if (isNaN(halfMoves) || halfMoves < 0)
300 |                 return {
301 |                     ok: !1,
302 |                     error: 'Invalid FEN: half move counter number must be a non-negative integer',
303 |                 };
304 |             if (!/^[^![]abcdefghijklmnopqrstuvwxyz0-9]$/ .test(tokens[3]))
305 |                 return { ok: !1, error: 'Invalid FEN: en-passant square is invalid' };
306 |             if (/^[^kKqQ-]$/ .test(tokens[2]))
307 |                 return { ok: !1, error: 'Invalid FEN: castling availability is invalid' };
308 |             if (!/^(w|b)$/ .test(tokens[1]))
309 |                 return { ok: !1, error: 'Invalid FEN: side-to-move is invalid' };
310 |             const rows = tokens[0].split('/');
311 |             if (8 !== rows.length)
312 |                 return { ok: !1, error: 'Invalid FEN: piece data does not contain 8 '/'-delimited rows' };
313 |             for (let i = 0; i < rows.length; i++) {
314 |                 let sumFields = 0,
315 |                     previousWasNumber = !1;
316 |                 for (let k = 0; k < rows[i].length; k++)
317 |                     if (isDigit(rows[i][k])) {
318 |                         if (previousWasNumber)
319 |                             return { ok: !1, error: 'Invalid FEN: piece data is invalid (consecutive number)' };
320 |                         ((sumFields += parseInt(rows[i][k], 10)), (previousWasNumber = !0));
321 |                     } else {
322 |                         if (!/^[^prnbqkPRNBQK]$/ .test(rows[i][k]))
323 |                             return { ok: !1, error: 'Invalid FEN: piece data is invalid (invalid piece)' };
324 |                         ((sumFields += 1), (previousWasNumber = !1));
325 |                     }
326 |                 if (8 !== sumFields)
327 |                     return {
328 |                         ok: !1,
329 |                         error: 'Invalid FEN: piece data is invalid (too many squares in rank)',
330 |                     };
331 |             }
332 |             if (('3' === tokens[3][1] && 'w' === tokens[1]) || ('6' === tokens[3][1] && 'b' === tokens[1]))
333 |                 return { ok: !1, error: 'Invalid FEN: illegal en-passant square' };
334 |             const kings = [
335 |                 { color: 'white', regex: /K/g },
336 |                 { color: 'black', regex: /k/g },
337 |             ];
338 |             for (const { color: color, regex: regex } of kings) {
339 |                 if (!regex.test(tokens[0]))
340 |                     return { ok: !1, error: `Invalid FEN: missing ${color} king` };

```

```

341 |         if ((tokens[0].match(regex) || []).length > 1)
342 |             return { ok: !1, error: `Invalid FEN: too many ${color} kings` };
343 |         }
344 |         return Array.from(rows[0] + rows[7]).some((char) => 'P' === char.toUpperCase())
345 |         ? { ok: !1, error: `Invalid FEN: some pawns are on the edge rows` }
346 |         : { ok: !0 };
347 |     })(fen);
348 |     if (!ok) throw new Error(error);
349 | }
350 | const position = tokens[0];
351 | let square = 0;
352 | this.clear();
353 | for (let i = 0; i < position.length; i++) {
354 |     const piece = position.charAt(i);
355 |     if ('/' === piece) square += 8;
356 |     else if (isDigit(piece)) square += parseInt(piece, 10);
357 |     else {
358 |         const color = piece < 'a' ? 'w' : 'b';
359 |         (this._put({ type: piece.toLowerCase(), color: color }, algebraic(square)), square++);
360 |     }
361 | }
362 | ((this._turn = tokens[1],
363 |   tokens[2].indexOf('K') > -1 && (this._castling.w |= BITS.KSIDE_CASTLE),
364 |   tokens[2].indexOf('Q') > -1 && (this._castling.w |= BITS.QSIDE_CASTLE),
365 |   tokens[2].indexOf('k') > -1 && (this._castling.b |= BITS.KSIDE_CASTLE),
366 |   tokens[2].indexOf('q') > -1 && (this._castling.b |= BITS.QSIDE_CASTLE),
367 |   (this._epSquare = '-' === tokens[3] ? -1 : 0x88[tokens[3]]),
368 |   (this._halfMoves = parseInt(tokens[4], 10)),
369 |   (this._moveNumber = parseInt(tokens[5], 10)),
370 |   (this._hash = this._computeHash()),
371 |   this._incPositionCount());
372 | )
373 | fen({ forceEnpassantSquare: forceEnpassantSquare = !1 } = {}) {
374 |     let empty = 0,
375 |         fen = '';
376 |     for (let i = 0x88.a8; i <= 0x88.h1; i++) {
377 |         if (this._board[i]) {
378 |             empty > 0 && ((fen += empty), (empty = 0));
379 |             const { color: color, type: piece } = this._board[i];
380 |             fen += 'w' === color ? piece.toUpperCase() : piece.toLowerCase();
381 |             } else empty++;
382 |             (i + 1) & 136 &&
383 |             (empty > 0 && (fen += empty), i !== 0x88.h1 && (fen += '/'), (empty = 0), (i += 8));
384 |         }
385 |         let castling = '';
386 |         (this._castling.w & BITS.KSIDE_CASTLE && (castling += 'K'),
387 |         this._castling.w & BITS.QSIDE_CASTLE && (castling += 'Q'),
388 |         this._castling.b & BITS.KSIDE_CASTLE && (castling += 'k'),
389 |         this._castling.b & BITS.QSIDE_CASTLE && (castling += 'q'),
390 |         (castling = castling || '-'));
391 |         let epSquare = '-';
392 |         if (-1 !== this._epSquare)
393 |             if (forceEnpassantSquare) epSquare = algebraic(this._epSquare);
394 |             else {
395 |                 const bigPawnSquare = this._epSquare + ('w' === this._turn ? 16 : -16),
396 |                     squares = [bigPawnSquare + 1, bigPawnSquare - 1];
397 |                 for (const square of squares) {
398 |                     if (136 & square) continue;
399 |                     const color = this._turn;
400 |                     if (this._board[square]?.color === color && 'p' === this._board[square]?.type) {
401 |                         this._makeMove({
402 |                             color: color,
403 |                             from: square,
404 |                             to: this._epSquare,
405 |                             piece: 'p',
406 |                             captured: 'p',
407 |                             flags: BITS.EP_CAPTURE,
408 |                         });
409 |                         const isLegal = !this._isKingAttacked(color);
410 |                         if ((this._undoMove(), isLegal)) {
411 |                             epSquare = algebraic(this._epSquare);

```



```

412 |         break;
413 |     }
414 | }
415 | }
416 | }
417 | return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
418 | }
419 | _pieceKey(i) {
420 |     if (!this._board[i]) return 0n;
421 |     const { color: color, type: type } = this._board[i],
422 |           typeIndex = { p: 0, n: 1, b: 2, r: 3, q: 4, k: 5 }[type];
423 |     return PIECE_KEYS[{ w: 0, b: 1 }[color]][typeIndex][i];
424 | }
425 | _epKey() {
426 |     return -1 === this._epSquare ? 0n : EP_KEYS[7 & this._epSquare];
427 | }
428 | _castlingKey() {
429 |     const index = (this._castling.w >> 5) | (this._castling.b >> 3);
430 |     return CASTLING_KEYS[index];
431 | }
432 | _computeHash() {
433 |     let hash = 0n;
434 |     for (let i = 0x88.a8; i <= 0x88.h1; i++)
435 |         136 & i ? (i += 7) : this._board[i] && (hash ^= this._pieceKey(i));
436 |     return (
437 |         (hash ^= this._epKey()),
438 |         (hash ^= this._castlingKey()),
439 |         'b' === this._turn && (hash ^= SIDE_KEY),
440 |         hash
441 |     );
442 | }
443 | reset() {
444 |     this.load(DEFAULT_POSITION);
445 | }
446 | get(square) {
447 |     return this._board[0x88[square]];
448 | }
449 | _set(sq, piece) {
450 |     ((this._hash ^= this._pieceKey(sq)),
451 |     (this._board[sq] = piece),
452 |     (this._hash ^= this._pieceKey(sq)));
453 | }
454 | put({ type: type, color: color }, square) {
455 |     if (-1 === 'pnbrqkPNBRQK'.indexOf(type.toLowerCase())) return !1;
456 |     if (!(square in 0x88)) return !1;
457 |     const sq = 0x88[square];
458 |     if ('k' === type && -1 !== this._kings[color] && this._kings[color] !== sq) return !1;
459 |     const currentPieceOnSquare = this._board[sq];
460 |     return (
461 |         currentPieceOnSquare &&
462 |         'k' === currentPieceOnSquare.type &&
463 |         (this._kings[currentPieceOnSquare.color] !== -1),
464 |         this._set(sq, { type: type, color: color }),
465 |         'k' === type && (this._kings[color] = sq),
466 |         !0
467 |     );
468 | }
469 | _clear(sq) {
470 |     ((this._hash ^= this._pieceKey(sq)), delete this._board[sq]);
471 | }
472 | _attacked(color, square, verbose) {
473 |     const attackers = [];
474 |     for (let i = 0x88.a8; i <= 0x88.h1; i++) {
475 |         if (136 & i) {
476 |             i += 7;
477 |             continue;
478 |         }
479 |         if (void 0 === this._board[i] || this._board[i].color !== color) continue;
480 |         const piece = this._board[i],
481 |               difference = i - square;
482 |         if (0 === difference) continue;

```

```

483 |         const index = difference + 119;
484 |         if (ATTACKS[index] & PIECE_MASKS[piece.type]) {
485 |             if ('p' === piece.type) {
486 |                 if ((difference > 0 && 'w' === piece.color) || (difference ≤ 0 && 'b' === piece.color)) {
487 |                     if (!verbose) return !0;
488 |                     attackers.push(algebraic(i));
489 |                 }
490 |                 continue;
491 |             }
492 |             if ('n' === piece.type || 'k' === piece.type) {
493 |                 if (verbose) {
494 |                     attackers.push(algebraic(i));
495 |                     continue;
496 |                 }
497 |                 return !0;
498 |             }
499 |             const offset = RAYS[index];
500 |             let j = i + offset,
501 |                 blocked = !1;
502 |             for (; j ≠ square; ) {
503 |                 if (null ≠ this._board[j]) {
504 |                     blocked = !0;
505 |                     break;
506 |                 }
507 |                 j += offset;
508 |             }
509 |             if (!blocked) {
510 |                 if (verbose) {
511 |                     attackers.push(algebraic(i));
512 |                     continue;
513 |                 }
514 |                 return !0;
515 |             }
516 |         }
517 |     }
518 |     return !!verbose && attackers;
519 | }
520 | attackers(square, attackedBy) {
521 |     return attackedBy
522 |     ? this._attacked(attackedBy, 0x88[square], !0)
523 |     : this._attacked(this._turn, 0x88[square], !0);
524 | }
525 | _isKingAttacked(color) {
526 |     const square = this._kings[color];
527 |     return -1 ≠ square && this._attacked(swapColor(color), square);
528 | }
529 | hash() {
530 |     return this._hash.toString(16);
531 | }
532 | isAttacked(square, attackedBy) {
533 |     return this._attacked(attackedBy, 0x88[square]);
534 | }
535 | isCheck() {
536 |     return this._isKingAttacked(this._turn);
537 | }
538 | inCheck() {
539 |     return this.isCheck();
540 | }
541 | isCheckmate() {
542 |     return this.isCheck() && 0 === this._moves().length;
543 | }
544 | isStalemate() {
545 |     return !this.isCheck() && 0 === this._moves().length;
546 | }
547 | isInsufficientMaterial() {
548 |     const pieces = {b: 0, n: 0, r: 0, q: 0, k: 0, p: 0},
549 |         bishops = [];
550 |     let numPieces = 0,
551 |         squareColor = 0;
552 |     for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
553 |         if (((squareColor = (squareColor + 1) % 2), 136 & i)) {

```

```

554 |         i += 7;
555 |         continue;
556 |     }
557 |     const piece = this._board[i];
558 |     piece &&
559 |     ((pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1),
560 |     'b' === piece.type && bishops.push(squareColor),
561 |     numPieces++);
562 | }
563 | if (2 === numPieces) return !0;
564 | if (3 === numPieces && (1 === pieces.b || 1 === pieces.n)) return !0;
565 | if (numPieces === pieces.b + 2) {
566 |     let sum = 0;
567 |     const len = bishops.length;
568 |     for (let i = 0; i < len; i++) sum += bishops[i];
569 |     if (0 === sum || sum === len) return !0;
570 | }
571 | return !1;
572 | }
573 | isThreefoldRepetition() {
574 |     return this._getPositionCount(this._hash) >= 3;
575 | }
576 | isDrawByFiftyMoves() {
577 |     return this._halfMoves >= 100;
578 | }
579 | isDraw() {
580 |     return (
581 |         this.isDrawByFiftyMoves() ||
582 |         this.isStalemate() ||
583 |         this.isInsufficientMaterial() ||
584 |         this.isThreefoldRepetition()
585 |     );
586 | }
587 | isGameOver() {
588 |     return this.isCheckmate() || this.isDraw();
589 | }
590 | moves({ verbose: verbose = !1, square: square, piece: piece } = {}) {
591 |     const moves = this._moves({ square: square, piece: piece });
592 |     return verbose
593 |         ? moves.map((move) => new Move(this, move))
594 |         : moves.map((move) => this._moveToSan(move, moves));
595 | }
596 | _moves({ legal: legal = !0, piece: piece, square: square } = {}) {
597 |     const forSquare = square ? square.toLowerCase() : void 0,
598 |         forPiece = piece ? piece.toLowerCase(),
599 |         moves = [],
600 |         us = this._turn,
601 |         them = swapColor(us);
602 |     let firstSquare = 0x88.a8,
603 |         lastSquare = 0x88.h1,
604 |         singleSquare = !1;
605 |     if (forSquare) {
606 |         if (!(forSquare in 0x88)) return [];
607 |         ((firstSquare = lastSquare = 0x88[forSquare]), (singleSquare = !0));
608 |     }
609 |     for (let from = firstSquare; from <= lastSquare; from++) {
610 |         if (136 & from) {
611 |             from += 7;
612 |             continue;
613 |         }
614 |         if (!this._board[from] || this._board[from].color === them) continue;
615 |         const { type: type } = this._board[from];
616 |         let to;
617 |         if ('p' === type) {
618 |             if (forPiece && forPiece !== type) continue;
619 |             ((to = from + PAWN_OFFSETS[us][0]),
620 |             this._board[to] ||
621 |             (addMove(moves, us, from, to, 'p'),
622 |             (to = from + PAWN_OFFSETS[us][1]),
623 |             SECOND_RANK[us] !== rank(from) ||
624 |             this._board[to] ||

```

```

625 |         addMove(moves, us, from, to, 'p', void 0, BITS.BIG_PAWN));
626 |         for (let j = 2; j < 4; j++)
627 |             ((to = from + PAWN_OFFSETS[us][j]),
628 |              136 & to) ||
629 |             (this._board[to]?.color === them
630 |              ? addMove(moves, us, from, to, 'p', this._board[to].type, BITS.CAPTURE)
631 |              : to === this._epSquare &
632 |                addMove(moves, us, from, to, 'p', 'p', BITS.EP_CAPTURE));
633 |     } else {
634 |         if (forPiece && forPiece !== type) continue;
635 |         for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++) {
636 |             const offset = PIECE_OFFSETS[type][j];
637 |             for (to = from; to += offset, !(136 & to);) {
638 |                 if (this._board[to]) {
639 |                     if (this._board[to].color === us) break;
640 |                     addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
641 |                     break;
642 |                 }
643 |                 if ((addMove(moves, us, from, to, type), 'n' === type || 'k' === type)) break;
644 |             }
645 |         }
646 |     }
647 | }
648 | if (
649 |     !(
650 |         (void 0 !== forPiece && 'k' !== forPiece) ||
651 |         (singleSquare && lastSquare !== this._kings[us])
652 |     )
653 | ) {
654 |     if (this._castling[us] & BITS.KSIDE_CASTLE) {
655 |         const castlingFrom = this._kings[us],
656 |             castlingTo = castlingFrom + 2;
657 |         this._board[castlingFrom + 1] ||
658 |         this._board[castlingTo] ||
659 |         this._attacked(them, this._kings[us]) ||
660 |         this._attacked(them, castlingFrom + 1) ||
661 |         this._attacked(them, castlingTo) ||
662 |         addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.KSIDE_CASTLE);
663 |     }
664 |     if (this._castling[us] & BITS.QSIDE_CASTLE) {
665 |         const castlingFrom = this._kings[us],
666 |             castlingTo = castlingFrom - 2;
667 |         this._board[castlingFrom - 1] ||
668 |         this._board[castlingFrom - 2] ||
669 |         this._board[castlingFrom - 3] ||
670 |         this._attacked(them, this._kings[us]) ||
671 |         this._attacked(them, castlingFrom - 1) ||
672 |         this._attacked(them, castlingTo) ||
673 |         addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.QSIDE_CASTLE);
674 |     }
675 | }
676 | if (!legal || -1 === this._kings[us]) return moves;
677 | const legalMoves = [];
678 | for (let i = 0, len = moves.length; i < len; i++)
679 |     (this._makeMove(moves[i]),
680 |      this._isKingAttacked(us) || legalMoves.push(moves[i]),
681 |      this._undoMove());
682 | return legalMoves;
683 | }
684 | move(move) {
685 |     let moveObj = null;
686 |     const moves = this._moves();
687 |     for (let i = 0, len = moves.length; i < len; i++)
688 |         if (
689 |             move.from === algebraic(moves[i].from) &&
690 |             move.to === algebraic(moves[i].to) &&
691 |             (!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
692 |         ) {
693 |             moveObj = moves[i];
694 |             break;
695 |         }

```

```

696 |     if (!moveObj) throw new Error(`Invalid move: ${JSON.stringify(move)}`);
697 |     const prettyMove = new Move(this, moveObj);
698 |     return (this._makeMove(moveObj), this._incPositionCount(), prettyMove);
699 | }
700 | _push(move) {
701 |     this._history.push({
702 |         move: move,
703 |         kings: { b: this._kings.b, w: this._kings.w },
704 |         turn: this._turn,
705 |         castling: { b: this._castling.b, w: this._castling.w },
706 |         epSquare: this._epSquare,
707 |         halfMoves: this._halfMoves,
708 |         moveNumber: this._moveNumber,
709 |     });
710 | }
711 | _movePiece(from, to) {
712 |     ((this._hash ^= this._pieceKey(from)),
713 |     (this._board[to] = this._board[from]),
714 |     delete this._board[from],
715 |     (this._hash ^= this._pieceKey(to)));
716 | }
717 | _makeMove(move) {
718 |     const us = this._turn,
719 |           them = swapColor(us);
720 |     if ((this._push(move), move.flags & BITS.NULL_MOVE))
721 |         return (
722 |             'b' === us && this._moveNumber++,
723 |             this._halfMoves++,
724 |             (this._turn = them),
725 |             void (this._epSquare = -1)
726 |         );
727 |     if (
728 |         ((this._hash ^= this._epKey()),
729 |         (this._hash ^= this._castlingKey()),
730 |         move.captured && (this._hash ^= this._pieceKey(move.to)),
731 |         this._movePiece(move.from, move.to),
732 |         move.flags & BITS.EP_CAPTURE &&
733 |         ('b' === this._turn ? this._clear(move.to - 16) : this._clear(move.to + 16)),
734 |         move.promotion &&
735 |         (this._clear(move.to), this._set(move.to, { type: move.promotion, color: us })),
736 |         'k' === this._board[move.to].type
737 |     ) {
738 |         if (((this._kings[us] = move.to), move.flags & BITS.KSIDE_CASTLE)) {
739 |             const castlingTo = move.to - 1,
740 |                   castlingFrom = move.to + 1;
741 |             this._movePiece(castlingFrom, castlingTo);
742 |         } else if (move.flags & BITS.QSIDE_CASTLE) {
743 |             const castlingTo = move.to + 1,
744 |                   castlingFrom = move.to - 2;
745 |             this._movePiece(castlingFrom, castlingTo);
746 |         }
747 |         this._castling[us] = 0;
748 |     }
749 |     if (this._castling[us])
750 |         for (let i = 0, len = ROOKS[us].length; i < len; i++)
751 |             if (move.from === ROOKS[us][i].square && this._castling[us] & ROOKS[us][i].flag) {
752 |                 this._castling[us] ^= ROOKS[us][i].flag;
753 |                 break;
754 |             }
755 |     if (this._castling[them])
756 |         for (let i = 0, len = ROOKS[them].length; i < len; i++)
757 |             if (move.to === ROOKS[them][i].square && this._castling[them] & ROOKS[them][i].flag) {
758 |                 this._castling[them] ^= ROOKS[them][i].flag;
759 |                 break;
760 |             }
761 |     if (((this._hash ^= this._castlingKey()), move.flags & BITS.BIG_PAWN)) {
762 |         let epSquare;
763 |         ((epSquare = 'b' === us ? move.to - 16 : move.to + 16),
764 |         ((move.to - 1) & 136 ||
765 |         'p' !== this._board[move.to - 1].type ||
766 |         this._board[move.to - 1].color !== them) &&

```

```

767 |         ((move.to + 1) < 136) ||
768 |         'p' !== this._board[move.to + 1]? .type ||
769 |         this._board[move.to + 1]? .color !== them)
770 |         ? (this._epSquare = -1)
771 |         : ((this._epSquare = epSquare), (this._hash ^= this._epKey()));
772 |     } else this._epSquare = -1;
773 |     ('p' === move.piece || move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)
774 |     ? (this._halfMoves = 0)
775 |     : this._halfMoves++,
776 |     'b' === us && this._moveNumber++,
777 |     (this._turn = them),
778 |     (this._hash ^= SIDE_KEY));
779 | }
780 | _undo() {
781 |     const hash = this._hash,
782 |           move = this._undoMove();
783 |     if (move) {
784 |         const prettyMove = new Move(this, move);
785 |         return (this._decPositionCount(hash), prettyMove);
786 |     }
787 |     return null;
788 | }
789 | _undoMove() {
790 |     const old = this._history.pop();
791 |     if (void 0 === old) return null;
792 |     ((this._hash ^= this._epKey()), (this._hash ^= this._castlingKey()));
793 |     const move = old.move;
794 |     ((this._kings = old.kings),
795 |     (this._turn = old.turn),
796 |     (this._castling = old.castling),
797 |     (this._epSquare = old.epSquare),
798 |     (this._halfMoves = old.halfMoves),
799 |     (this._moveNumber = old.moveNumber),
800 |     (this._hash ^= this._epKey()),
801 |     (this._hash ^= this._castlingKey()),
802 |     (this._hash ^= SIDE_KEY));
803 |     const us = this._turn,
804 |           them = swapColor(us);
805 |     if (move.flags & BITS.NULL_MOVE) return move;
806 |     if (
807 |         (this._movePiece(move.to, move.from),
808 |         move.piece && (this._clear(move.from), this._set(move.from, { type: move.piece, color: us })),
809 |         move.captured)
810 |     )
811 |         if (move.flags & BITS.EP_CAPTURE) {
812 |             let index;
813 |             ((index = 'b' === us ? move.to - 16 : move.to + 16),
814 |             this._set(index, { type: 'p', color: them }));
815 |         } else this._set(move.to, { type: move.captured, color: them });
816 |     if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
817 |         let castlingTo, castlingFrom;
818 |         (move.flags & BITS.KSIDE_CASTLE
819 |         ? ((castlingTo = move.to + 1), (castlingFrom = move.to - 1))
820 |         : ((castlingTo = move.to - 2), (castlingFrom = move.to + 1)),
821 |         this._movePiece(castlingFrom, castlingTo));
822 |     }
823 |     return move;
824 | }
825 | _moveToSan(move, moves) {
826 |     let output = '';
827 |     if (move.flags & BITS.KSIDE_CASTLE) output = 'O-O';
828 |     else if (move.flags & BITS.QSIDE_CASTLE) output = 'O-O-O';
829 |     else {
830 |         if ('p' !== move.piece) {
831 |             const disambiguator = (function getDisambiguator(move, moves) {
832 |                 const from = move.from,
833 |                       to = move.to,
834 |                       piece = move.piece;
835 |                 let ambiguities = 0,
836 |                     sameRank = 0,
837 |                     sameFile = 0;

```

```

838 |         for (let i = 0, len = moves.length; i < len; i++) {
839 |             const ambigFrom = moves[i].from,
840 |                   ambigTo = moves[i].to;
841 |             piece === moves[i].piece &&
842 |             from === ambigFrom &&
843 |             to === ambigTo &&
844 |             (ambiguities++,
845 |              rank(from) === rank(ambigFrom) && sameRank++,
846 |              file(from) === file(ambigFrom) && sameFile++);
847 |         }
848 |         return ambiguities > 0
849 |             ? sameRank > 0 && sameFile > 0
850 |             : algebraic(from)
851 |             : sameFile > 0
852 |             ? algebraic(from).charAt(1)
853 |             : algebraic(from).charAt(0)
854 |             : '';
855 |     })(move, moves);
856 |     output += move.piece.toUpperCase() + disambiguator;
857 | }
858 | (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE) &&
859 | ('p' === move.piece && (output += algebraic(move.from)[0]), (output += 'x')),
860 | (output += algebraic(move.to)),
861 | move.promotion && (output += '=' + move.promotion.toUpperCase()));
862 | }
863 | return (
864 |     this._makeMove(move),
865 |     this.isCheck() && (this.isCheckmate() ? (output += '#') : (output += '+')),
866 |     this._undoMove(),
867 |     output
868 | );
869 | }
870 | turn() {
871 |     return this._turn;
872 | }
873 | board() {
874 |     const output = [];
875 |     let row = [];
876 |     for (let i = 0x88.a8; i ≤ 0x88.h1; i++)
877 |         (null === this._board[i]
878 |         ? row.push(null)
879 |         : row.push({
880 |             square: algebraic(i),
881 |             type: this._board[i].type,
882 |             color: this._board[i].color,
883 |         })),
884 |         (i + 1) & 136 && (output.push(row), (row = []), (i += 8));
885 |     return output;
886 | }
887 | squareColor(square) {
888 |     if (square in 0x88) {
889 |         const sq = 0x88[square];
890 |         return (rank(sq) + file(sq)) % 2 === 0 ? 'light' : 'dark';
891 |     }
892 |     return null;
893 | }
894 | history({ verbose: verbose = !1 } = {}) {
895 |     const reversedHistory = [],
896 |           moveHistory = [];
897 |     for (; this._history.length > 0; ) reversedHistory.push(this._undoMove());
898 |     for (;;) {
899 |         const move = reversedHistory.pop();
900 |         if (!move) break;
901 |         (verbose
902 |         ? moveHistory.push(new Move(this, move))
903 |         : moveHistory.push(this._moveToSan(move, this._moves()))),
904 |         this._makeMove(move);
905 |     }
906 |     return moveHistory;
907 | }
908 | _getPositionCount(hash) {

```

```

909 |     return this._positionCount.get(hash) ?? 0;
910 | }
911 | _incPositionCount() {
912 |     this._positionCount.set(this._hash, (this._positionCount.get(this._hash) ?? 0) + 1);
913 | }
914 | _decPositionCount(hash) {
915 |     const currentCount = this._positionCount.get(hash) ?? 0;
916 |     1 === currentCount
917 |     ? this._positionCount.delete(hash)
918 |     : this._positionCount.set(hash, currentCount - 1);
919 | }
920 | moveNumber() {
921 |     return this._moveNumber;
922 | }
923 | }
924 | const AI_PRESETS = {
925 |     fast: { preset: 'fast', maxDepth: 2, timeLimitMs: 750, randomness: 20, quiescence: !0 },
926 |     balanced: { preset: 'balanced', maxDepth: 4, timeLimitMs: 2500, randomness: 8, quiescence: !0 },
927 |     strong: { preset: 'strong', maxDepth: 5, timeLimitMs: 5e3, randomness: 0, quiescence: !0 },
928 | },
929 | DEFAULT_AI_SETTINGS = { ...AI_PRESETS.strong };
930 | function clamp(value, min, max) {
931 |     return Math.min(max, Math.max(min, value));
932 | }
933 | const BOARD_FILES = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'],
934 |     BLACK_FILES = [...BOARD_FILES].reverse(),
935 |     WHITE_RANKS = [8, 7, 6, 5, 4, 3, 2, 1],
936 |     BLACK_RANKS = [...WHITE_RANKS].reverse(),
937 |     PROMOTION_PIECES = ['q', 'r', 'b', 'n'],
938 |     SQUARE_SET = new Set([
939 |         'a8',
940 |         'b8',
941 |         'c8',
942 |         'd8',
943 |         'e8',
944 |         'f8',
945 |         'g8',
946 |         'h8',
947 |         'a7',
948 |         'b7',
949 |         'c7',
950 |         'd7',
951 |         'e7',
952 |         'f7',
953 |         'g7',
954 |         'h7',
955 |         'a6',
956 |         'b6',
957 |         'c6',
958 |         'd6',
959 |         'e6',
960 |         'f6',
961 |         'g6',
962 |         'h6',
963 |         'a5',
964 |         'b5',
965 |         'c5',
966 |         'd5',
967 |         'e5',
968 |         'f5',
969 |         'g5',
970 |         'h5',
971 |         'a4',
972 |         'b4',
973 |         'c4',
974 |         'd4',
975 |         'e4',
976 |         'f4',
977 |         'g4',
978 |         'h4',
979 |         'a3',

```



```

980 | | | 'b3',
981 | | | 'c3',
982 | | | 'd3',
983 | | | 'e3',
984 | | | 'f3',
985 | | | 'g3',
986 | | | 'h3',
987 | | | 'a2',
988 | | | 'b2',
989 | | | 'c2',
990 | | | 'd2',
991 | | | 'e2',
992 | | | 'f2',
993 | | | 'g2',
994 | | | 'h2',
995 | | | 'a1',
996 | | | 'b1',
997 | | | 'c1',
998 | | | 'd1',
999 | | | 'e1',
1000 | | | 'f1',
1001 | | | 'g1',
1002 | | | 'h1',
1003 | | | ]),
1004 | | | CLOCK_PRESETS = {
1005 | | | | | '1/1': { baseMs: 6e4, incrementMs: 1e3 },
1006 | | | | | '3/2': { baseMs: 18e4, incrementMs: 2e3 },
1007 | | | | | '5/3': { baseMs: 3e5, incrementMs: 3e3 },
1008 | | | | | '10/5': { baseMs: 6e5, incrementMs: 5e3 },
1009 | | | | | '15/10': { baseMs: 9e5, incrementMs: 1e4 },
1010 | | | | },
1011 | | | | PIECE_GLYPHS = {
1012 | | | | | w: { p: '△', n: '△', b: '▲', r: '▣', q: '⊕', k: '⊙' },
1013 | | | | | b: { p: '▲', n: '▲', b: '▲', r: '▣', q: '⊕', k: '⊙' },
1014 | | | | },
1015 | | | function requireElement(selector, root) {
1016 | | | | const element = root.querySelector(selector);
1017 | | | | if (!element) throw new Error(`Missing element: ${selector}`);
1018 | | | | return element;
1019 | | | }
1020 | | | function createCoordinate(type, label) {
1021 | | | | const coordinate = document.createElement('span');
1022 | | | | return (
1023 | | | | | (coordinate.className = `coordinate coordinate-${type}`),
1024 | | | | | (coordinate.textContent = label),
1025 | | | | | coordinate
1026 | | | | );
1027 | | | }
1028 | | | function toSquare(file2, rank2) {
1029 | | | | return `${file2}${rank2}`;
1030 | | | }
1031 | | | function createClockState(preset) {
1032 | | | | const baseMs = CLOCK_PRESETS[preset].baseMs;
1033 | | | | return { w: baseMs, b: baseMs };
1034 | | | }
1035 | | | function colorName(color) {
1036 | | | | return 'w' === color ? '백' : '흑';
1037 | | | }
1038 | | | function oppositeColor(color) {
1039 | | | | return 'w' === color ? 'b' : 'w';
1040 | | | }
1041 | | | function getSquareLabel(square, piece) {
1042 | | | | return piece ? `${square}.${colorName(piece.color)}.${pieceName(piece.type)}` : `${square}.빈칸`;
1043 | | | }
1044 | | | function pieceName(piece) {
1045 | | | | switch (piece) {
1046 | | | | | case 'p':
1047 | | | | | return '폰';
1048 | | | | | case 'n':
1049 | | | | | return '나이트';
1050 | | | | | case 'b':

```

```

1051 |     return '비숍';
1052 |     case 'r':
1053 |     return '룩';
1054 |     case 'q':
1055 |     return '퀸';
1056 |     case 'k':
1057 |     return '킹';
1058 | }
1059 | }
1060 | !(function createChessApp() {
1061 |     const root = document.createElement('main');
1062 |     ((root.className = 'chess-app'),
1063 |     (root.innerHTML =
1064 |     `
        <aside class="control-panel" aria-label="게임 컨트롤">\n
        <header class="app-header">\n
        <p class="eyebro
        w">Airgap Chess</p>\n
        <h1>TS Chess</h1>\n
        </header>\n
        <section class="control-group" aria-label="모드">\n
        <span class="control-label">모드</span>\n
        <div class="segmented-control" role="group" aria-label="플레이 모드">\n
        <button class="segmented-button" type="button" data-mode="local">로컬 2인</button>\n
        <button class="segm
        ented-button" type="button" data-mode="ai">AI 상대</button>\n
        </div>\n
        </section>\n
        <section id="color-c
        ontrols" class="control-group" aria-label="진영">\n
        <span class="control-label">진영</span>\n
        <div class="segme
        nted-control" role="group" aria-label="인간 진영">\n
        <button class="segmented-button" type="button" data-color="w">백
        </button>\n
        <button class="segmented-button" type="button" data-color="b">흑</button>\n
        </div>\n
        </se
        ction>\n
        <section id="ai-controls" class="control-group ai-settings" aria-label="AI 설정">\n
        <span class="con
        trol-label">AI 설정</span>\n
        <div class="segmented-control segmented-control-three" role="group" aria-label="AI 프리셋"
        >\n
        <button class="segmented-button" type="button" data-preset="fast">빠름</button>\n
        <button class="segm
        ented-button" type="button" data-preset="balanced">균형</button>\n
        <button class="segmented-button" type="button"
        data-preset="strong">최강</button>\n
        </div>\n
        <label class="range-control">\n
        <span>최대 깊이<strong id
        ="ai-depth-value"></strong></span>\n
        <input id="ai-depth-input" type="range" min="1" max="5" step="1" />\n
        </label>\n
        <label class="range-control">\n
        <span>시간 제한<strong id="ai-time-value"></strong></span>\n
        <input id="ai-time-input" type="range" min="500" max="5000" step="250" />\n
        </label>\n
        <label class=
        "range-control">\n
        <span>수 다양성<strong id="ai-randomness-value"></strong></span>\n
        <input
        id="ai-randomness-input" type="range" min="0" max="30" step="1"
        aria-label="최선 후보 근처에서 다양한 수를 고르는 정도">\n
        <input id="ai-quiescence-input" type="checkbox" />\n
        <span
        >전술 탐색</span>\n
        </label>\n
        </section>\n
        <section class="control-group clock-settings" aria-label="시간 설
        정">\n
        <span class="control-label">시간</span>\n
        <div class="clock-preset-control" role="group" aria-label="시간
        프리셋">\n
        <button class="segmented-button" type="button" data-clock-preset="1/1">1/1</button>\n
        <button
        class="segmented-button" type="button" data-clock-preset="3/2">3/2</button>\n
        <button class="segmented-button" t
        ype="button" data-clock-preset="5/3">5/3</button>\n
        <button class="segmented-button" type="button" data-clock-pr
        eset="10/5">10/5</button>\n
        <button class="segmented-button" type="button" data-clock-preset="15/10">15/10</butt
        on>\n
        </div>\n
        </section>\n
        <section class="action-row" aria-label="게임 동작">\n
        <button id="new-g
        ame-button" class="primary-button" type="button">새 게임</button>\n
        <button id="undo-button" class="ghost-button" typ
        e="button">되돌리기</button>\n
        </section>\n
        </aside>\n
        <section class="board-stage" aria-label="체스판">\n
        <di
        v id="chess-board" class="chess-board" role="grid" aria-label="체스판"></div>\n
        </section>\n
        <aside class="clock-pan
        el" aria-label="체스 타이머">\n
        <div id="clock-display" class="clock-display">\n
        <div id="white-clock" class="cloc
        k-card">\n
        <span class="clock-side">백</span>\n
        <strong id="white-clock-time" class="clock-time"></strong>\n
        </div>\n
        <div id="black-clock" class="clock-card">\n
        <span class="clock-side">흑</span>\n
        <strong id="black-clock-time" class="clock-time"></strong>\n
        </div>\n
        </div>\n
        </aside>\n
        <aside
        class="info-panel" aria-label="게임 정보">\n
        <section class="status-panel" aria-live="polite">\n
        <span class="co
        ntrol-label">상태</span>\n
        <strong id="game-status" class="game-status"></strong>\n
        <dl class="status-grid">\n
        <div>\n
        <dt>차례</dt>\n
        <dd id="turn-meta"></dd>\n
        </div>\n
        <div>\n
        <dt>모드</dt>\n
        <dd id="mode-meta"></dd>\n
        </div>\n
        </div>\n
        <div>\n
        <dt>수순</dt>\n
        <dd id="move-meta"></dd>\n
        </div>\n
        </dl>\n
        </section>\n
        <section class="history-pan
        el" aria-label="기보">\n
        <div class="history-header">\n
        <span class="control-label">기보</span>\n
        </di
        v>\n
        <div id="move-list" class="move-list"></div>\n
        </section>\n
        </aside>\n
        <div id="promotion-overla
        y" class="promotion-overlay" hidden>\n
        <div class="promotion-dialog" role="dialog" aria-modal="true" aria-labelledby=
        "promotion-title">\n
        <h2 id="promotion-title">승격</h2>\n
        <div id="promotion-choices" class="promotion-choic
        es" role="group" aria-label="승격 기물"></div>\n
        </div>\n
        </div>\n
        </div>`
    ),
1065 |     document.body.replaceChildren(root));
1066 |     const boardElement = requireElement('#chess-board', root),
1067 |     colorControlsElement = requireElement('#color-controls', root),
1068 |     aiControlsElement = requireElement('#ai-controls', root),
1069 |     statusElement = requireElement('#game-status', root),
1070 |     turnMetaElement = requireElement('#turn-meta', root),
1071 |     modeMetaElement = requireElement('#mode-meta', root),
1072 |     moveMetaElement = requireElement('#move-meta', root),
1073 |     moveListElement = requireElement('#move-list', root),
1074 |     clockDisplayElement = requireElement('#clock-display', root),
1075 |     whiteClockElement = requireElement('#white-clock', root),
1076 |     blackClockElement = requireElement('#black-clock', root),
1077 |     whiteClockTimeElement = requireElement('#white-clock-time', root),
1078 |     blackClockTimeElement = requireElement('#black-clock-time', root),

```

```

1079 |   newGameButton = requireElement('#new-game-button', root),
1080 |   undoButton = requireElement('#undo-button', root),
1081 |   aiDepthInput = requireElement('#ai-depth-input', root),
1082 |   aiDepthValue = requireElement('#ai-depth-value', root),
1083 |   aiTimeInput = requireElement('#ai-time-input', root),
1084 |   aiTimeValue = requireElement('#ai-time-value', root),
1085 |   aiRandomnessInput = requireElement('#ai-randomness-input', root),
1086 |   aiRandomnessValue = requireElement('#ai-randomness-value', root),
1087 |   aiQuiescenceInput = requireElement('#ai-quiescence-input', root),
1088 |   promotionOverlay = requireElement('#promotion-overlay', root),
1089 |   promotionChoices = requireElement('#promotion-choices', root),
1090 |   modeButtons = Array.from(root.querySelectorAll('[data-mode]')),
1091 |   colorButtons = Array.from(root.querySelectorAll('[data-color]')),
1092 |   presetButtons = Array.from(root.querySelectorAll('[data-preset]')),
1093 |   clockPresetButtons = Array.from(root.querySelectorAll('[data-clock-preset]')),
1094 |   game = new Chess();
1095 |   let mode = 'local',
1096 |   clockPreset = '10/5',
1097 |   clockMs = createClockState(clockPreset),
1098 |   activeClockColor = null,
1099 |   lastClockTick = Date.now(),
1100 |   clockTimerId = null,
1101 |   timedOutColor = null,
1102 |   humanColor = 'w',
1103 |   selectedSquare = null,
1104 |   legalMoves = [],
1105 |   pendingPromotion = null,
1106 |   aiThinking = !1,
1107 |   aiTimerId = null,
1108 |   aiWorker = null,
1109 |   aiRequestId = 0,
1110 |   activeAiRequestId = null,
1111 |   activeAiFen = null,
1112 |   aiSettings = { ... DEFAULT_AI_SETTINGS },
1113 |   aiProgress = null;
1114 |   function selectSquare(square) {
1115 |     ((selectedSquare = square),
1116 |     ((legalMoves = game.moves({ verbose: !0, square: square }))),
1117 |     render());
1118 |   }
1119 |   function playMove(moveCommand) {
1120 |     if (!commitMove(moveCommand)) return (clearSelection(), void render());
1121 |     ((pendingPromotion = null), clearSelection(), render(), scheduleAiIfNeeded());
1122 |   }
1123 |   function resetGame() {
1124 |     (cancelAiMove(),
1125 |     game.reset(),
1126 |     (pendingPromotion = null),
1127 |     clearSelection(),
1128 |     resetClockState(),
1129 |     restartClockForCurrentTurn(),
1130 |     render(),
1131 |     scheduleAiIfNeeded());
1132 |   }
1133 |   function scheduleAiIfNeeded() {
1134 |     'ai' !== mode ||
1135 |     game.isGameOver() ||
1136 |     null !== timedOutColor ||
1137 |     game.turn() !== humanColor ||
1138 |     aiThinking ||
1139 |     pendingPromotion ||
1140 |     ((aiThinking = !0),
1141 |     (aiProgress = null),
1142 |     render(),
1143 |     (aiTimerId = window.setTimeout(() => {
1144 |       ((aiTimerId = null),
1145 |       (function startAiSearch() {
1146 |         aiWorker?.terminate();
1147 |         const requestId = aiRequestId + 1,
1148 |         fen = game.fen();
1149 |         ((aiRequestId = requestId),

```

```

1150 |         (activeAiRequestId = requestId),
1151 |         (activeAiFen = fen),
1152 |         (aiWorker = new Worker(new URL('/assets/ai.worker-B7h8TIdl.js', import.meta.url), {
1153 |             type: 'module',
1154 |         })),
1155 |         aiWorker.addEventListener('message', (event) => {
1156 |             !(function handleAiMessage(message) {
1157 |                 message.id === activeAiRequestId &&
1158 |                 null !== activeAiFen &&
1159 |                 game.fen() === activeAiFen &&
1160 |                 ((aiProgress = message),
1161 |                 'progress' !== message.kind
1162 |                 ? ((aiThinking = !1),
1163 |                 (activeAiRequestId = null),
1164 |                 (activeAiFen = null),
1165 |                 aiWorker?.terminate(),
1166 |                 (aiWorker = null),
1167 |                 message.move && commitMove(message.move),
1168 |                 clearSelection(),
1169 |                 render())
1170 |                 : renderStatus());
1171 |             })(event.data);
1172 |         }),
1173 |         aiWorker.addEventListener('error', () => {
1174 |             activeAiRequestId === requestId &&
1175 |             ((aiThinking = !1),
1176 |             (activeAiRequestId = null),
1177 |             (activeAiFen = null),
1178 |             aiWorker?.terminate(),
1179 |             (aiWorker = null),
1180 |             render());
1181 |         }),
1182 |         aiWorker.postMessage({ id: requestId, fen: fen, settings: aiSettings }));
1183 |     })());
1184 | }, 180));
1185 | }
1186 | function cancelAiMove() {
1187 |     (aiRequestId += 1),
1188 |     (activeAiRequestId = null),
1189 |     (activeAiFen = null),
1190 |     (aiProgress = null),
1191 |     null !== aiTimerId && (window.clearTimeout(aiTimerId), (aiTimerId = null)),
1192 |     aiWorker && (aiWorker.terminate(), (aiWorker = null)),
1193 |     (aiThinking = !1);
1194 | }
1195 | function commitMove(moveCommand) {
1196 |     const movingColor = game.turn();
1197 |     if (!flushClockElapsed()) return !1;
1198 |     try {
1199 |         game.move(moveCommand);
1200 |     } catch {
1201 |         return !1;
1202 |     }
1203 |     return (
1204 |         (clockMs[movingColor] += CLOCK_PRESETS[clockPreset].incrementMs),
1205 |         restartClockForCurrentTurn(),
1206 |         !0
1207 |     );
1208 | }
1209 | function updateAiSettings(settings) {
1210 |     ((aiSettings = (function normalizeAiSettings(settings) {
1211 |         return {
1212 |             preset: settings.preset,
1213 |             maxDepth: ((depth = settings.maxDepth), clamp(Math.round(depth), 1, 5)),
1214 |             timeLimitMs: clamp(Math.round(settings.timeLimitMs), 500, 5e3),
1215 |             randomness: clamp(Math.round(settings.randomness), 0, 30),
1216 |             quiescence: settings.quiescence,
1217 |         };
1218 |         var depth;
1219 |     }))(settings)),
1220 |     aiThinking && cancelAiMove(),

```

```

1221 | | | | | render(),
1222 | | | | | scheduleAiIfNeeded());
1223 | | | }
1224 | | | function resetClockState() {
1225 | | | | | ((clockMs = createClockState(clockPreset)),
1226 | | | | | (timedOutColor = null),
1227 | | | | | (activeClockColor = null),
1228 | | | | | (lastClockTick = Date.now()));
1229 | | | }
1230 | | | function restartClockForCurrentTurn() {
1231 | | | | | if ((clearClockTimer(), game.isGameOver() || null !== timedOutColor))
1232 | | | | | return ((activeClockColor = null), void renderClocks());
1233 | | | | | ((activeClockColor = game.turn()),
1234 | | | | | (lastClockTick = Date.now()),
1235 | | | | | (clockTimerId = window.setInterval(tickClock, 100)),
1236 | | | | | renderClocks());
1237 | | | }
1238 | | | function clearClockTimer() {
1239 | | | | | null !== clockTimerId && (window.clearInterval(clockTimerId), (clockTimerId = null));
1240 | | | }
1241 | | | function tickClock() {
1242 | | | | | flushClockElapsed() && renderClocks();
1243 | | | }
1244 | | | function flushClockElapsed() {
1245 | | | | | if (null === activeClockColor || null !== timedOutColor || game.isGameOver()) return !0;
1246 | | | | | const now = Date.now(),
1247 | | | | | elapsedMs = Math.max(0, now - lastClockTick);
1248 | | | | | return (
1249 | | | | | (lastClockTick = now),
1250 | | | | | 0 === elapsedMs ||
1251 | | | | | ((clockMs[activeClockColor] = Math.max(0, clockMs[activeClockColor] - elapsedMs)),
1252 | | | | | 0 !== clockMs[activeClockColor] ||
1253 | | | | | ((function handleClockTimeout(color) {
1254 | | | | | | | ((timedOutColor = color),
1255 | | | | | | | (activeClockColor = null),
1256 | | | | | | | (clockMs[color] = 0),
1257 | | | | | | | clearClockTimer(),
1258 | | | | | | | cancelAiMove(),
1259 | | | | | | | clearSelection(),
1260 | | | | | | | render());
1261 | | | | | | })(activeClockColor),
1262 | | | | | | !1))
1263 | | | | | );
1264 | | | }
1265 | | | function clearSelection() {
1266 | | | | | ((selectedSquare = null), (legalMoves = []));
1267 | | | }
1268 | | | function render() {
1269 | | | | | (!function renderControls() {
1270 | | | | | | | (modeButtons.forEach((button) => {
1271 | | | | | | | | | const isActive = button.dataset.mode === mode;
1272 | | | | | | | | | (button.classList.toggle('is-active', isActive),
1273 | | | | | | | | | button.setAttribute('aria-pressed', String(isActive)));
1274 | | | | | | | })),
1275 | | | | | | | (colorButtons.forEach((button) => {
1276 | | | | | | | | | const isActive = button.dataset.color === humanColor;
1277 | | | | | | | | | (button.classList.toggle('is-active', isActive),
1278 | | | | | | | | | button.setAttribute('aria-pressed', String(isActive)));
1279 | | | | | | | })),
1280 | | | | | | | (colorControlsElement.hidden = 'ai' !== mode),
1281 | | | | | | | (aiControlsElement.hidden = 'ai' !== mode),
1282 | | | | | | | (function renderAiSettingsControls() {
1283 | | | | | | | | | (presetButtons.forEach((button) => {
1284 | | | | | | | | | | | const isActive = button.dataset.preset === aiSettings.preset;
1285 | | | | | | | | | | | (button.classList.toggle('is-active', isActive),
1286 | | | | | | | | | | | button.setAttribute('aria-pressed', String(isActive)));
1287 | | | | | | | | | })),
1288 | | | | | | | | | (aiDepthInput.value = String(aiSettings.maxDepth)),
1289 | | | | | | | | | (aiDepthValue.textContent = `${aiSettings.maxDepth}`),
1290 | | | | | | | | | (aiTimeInput.value = String(aiSettings.timeLimitMs)),
1291 | | | | | | | | | (aiTimeValue.textContent = `${(function formatSeconds(milliseconds) {

```

```

1292 |         return `${(milliseconds / 1e3).toFixed(milliseconds % 1e3 === 0 ? 0 : 2)}초`;
1293 |     })(aiSettings.timeLimitMs)`),
1294 |     (aiRandomnessInput.value = String(aiSettings.randomness)),
1295 |     (aiRandomnessValue.textContent = `${aiSettings.randomness}/30`),
1296 |     (aiQuiescenceInput.checked = aiSettings.quiescence));
1297 |     })(),
1298 |     (function renderClockSettingsControls() {
1299 |         clockPresetButtons.forEach((button) => {
1300 |             const isActive = button.dataset.clockPreset === clockPreset;
1301 |             (button.classList.toggle('is-active', isActive),
1302 |             button.setAttribute('aria-pressed', String(isActive)));
1303 |         });
1304 |     })(),
1305 |     (undoButton.disabled = 0 === game.history().length));
1306 |     })(),
1307 |     renderClocks(),
1308 |     (function renderBoard() {
1309 |         const fragment = document.createDocumentFragment(),
1310 |             orientation = getBoardOrientation(),
1311 |             files = 'w' === orientation ? BOARD_FILES : BLACK_FILES,
1312 |             ranks = 'w' === orientation ? WHITE_RANKS : BLACK_RANKS,
1313 |             lastMove = (function getLastMove() {
1314 |                 return game.history({ verbose: !0 }).at(-1) ?? null;
1315 |             })();
1316 |         boardElement.classList.toggle('is-locked', !canUseBoard());
1317 |         for (const rank2 of ranks)
1318 |             for (const file2 of files) {
1319 |                 const square = toSquare(file2, rank2),
1320 |                     piece = game.get(square),
1321 |                     button = document.createElement('button');
1322 |                 if (
1323 |                     ((button.type = 'button'),
1324 |                     (button.className = getSqaureClasses(square, lastMove)),
1325 |                     (button.dataset.square = square),
1326 |                     (button.disabled = !canUseBoard()),
1327 |                     button.setAttribute('role', 'gridcell'),
1328 |                     button.setAttribute('aria-label', getSqaureLabel(square, piece)),
1329 |                     file2 === files[0] && button.appendChild(createCoordinate('rank', String(rank2))),
1330 |                     rank2 === ranks[ranks.length - 1] &&
1331 |                     button.appendChild(createCoordinate('file', file2)),
1332 |                     piece)
1333 |                 ) {
1334 |                     const pieceElement = document.createElement('span');
1335 |                     ((pieceElement.className = 'piece',
1336 |                     (pieceElement.textContent = PIECE_GLYPHS[piece.color][piece.type]),
1337 |                     button.appendChild(pieceElement));
1338 |                 }
1339 |                 fragment.appendChild(button);
1340 |             }
1341 |         boardElement.replaceChildren(fragment);
1342 |     })(),
1343 |     renderStatus(),
1344 |     (function renderMoveList() {
1345 |         const history = game.history({ verbose: !0 }),
1346 |             fragment = document.createDocumentFragment();
1347 |         if (0 === history.length) {
1348 |             const empty = document.createElement('p');
1349 |             return (
1350 |                 (empty.className = 'move-empty'),
1351 |                 (empty.textContent = '아직 둔 수가 없습니다.'),
1352 |                 fragment.appendChild(empty),
1353 |                 void moveListElement.replaceChildren(fragment)
1354 |             );
1355 |         }
1356 |         for (let index = 0; index < history.length; index += 2) {
1357 |             const row = document.createElement('div'),
1358 |                 moveNumber = document.createElement('span'),
1359 |                 whiteMove = document.createElement('span'),
1360 |                 blackMove = document.createElement('span');
1361 |             ((row.className = 'move-row',
1362 |             (moveNumber.className = 'move-number'),

```

```

1363 | | | | | (whiteMove.className = 'move-san'),
1364 | | | | | (blackMove.className = 'move-san'),
1365 | | | | | (moveNumber.textContent = `${Math.floor(index / 2) + 1}.`),
1366 | | | | | (whiteMove.textContent = history[index] ? .san ?? ''),
1367 | | | | | (blackMove.textContent = history[index + 1] ? .san ?? ''),
1368 | | | | | row.append(moveNumber, whiteMove, blackMove),
1369 | | | | | fragment.appendChild(row));
1370 | | | | | }
1371 | | | | | (moveListElement.replaceChildren(fragment),
1372 | | | | | (moveListElement.scrollTop = moveListElement.scrollHeight));
1373 | | | | | }()),
1374 | | | | | (function renderPromotionDialog() {
1375 | | | | | if (
1376 | | | | | ((promotionOverlay.hidden = null === pendingPromotion),
1377 | | | | | promotionChoices.replaceChildren(),
1378 | | | | | pendingPromotion)
1379 | | | | | )
1380 | | | | | for (const piece of PROMOTION_PIECES) {
1381 | | | | | const button = document.createElement('button');
1382 | | | | | ((button.className = 'promotion-button'),
1383 | | | | | (button.type = 'button'),
1384 | | | | | (button.dataset.promotion = piece),
1385 | | | | | button.setAttribute('aria-label', `${pieceName(piece)} 승격`),
1386 | | | | | (button.textContent = PIECE_GLYPHS[game.turn()][piece]),
1387 | | | | | promotionChoices.appendChild(button));
1388 | | | | | }
1389 | | | | | }()) );
1390 | | | }
1391 | | function renderClocks() {
1392 | | const bottomColor = getBoardOrientation(),
1393 | | topClockElement = 'w' === oppositeColor(bottomColor) ? whiteClockElement : blackClockElement,
1394 | | bottomClockElement = 'w' === bottomColor ? whiteClockElement : blackClockElement;
1395 | | ((clockDisplayElement.children[0] === topClockElement &&
1396 | | clockDisplayElement.children[1] === bottomClockElement) ||
1397 | | clockDisplayElement.replaceChildren(topClockElement, bottomClockElement),
1398 | | renderClockPosition('w', bottomColor),
1399 | | renderClockPosition('b', bottomColor),
1400 | | renderClock('w', whiteClockElement, whiteClockTimeElement),
1401 | | renderClock('b', blackClockElement, blackClockTimeElement));
1402 | | }
1403 | | function renderClockPosition(color, bottomColor) {
1404 | | const clockElement = 'w' === color ? whiteClockElement : blackClockElement,
1405 | | isBottom = color === bottomColor;
1406 | | (clockElement.classList.toggle('is-bottom', isBottom),
1407 | | clockElement.classList.toggle('is-top', !isBottom));
1408 | | }
1409 | | function renderClock(color, clockElement, timeElement) {
1410 | | const remainingMs = clockMs[color],
1411 | | isActive = activeClockColor === color && null === timedOutColor,
1412 | | isTimedOut = timedOutColor === color;
1413 | | (clockElement.classList.toggle('is-active', isActive),
1414 | | clockElement.classList.toggle('is-warning', remainingMs ≤ 3e4),
1415 | | clockElement.classList.toggle('is-danger', remainingMs ≤ 1e4),
1416 | | clockElement.classList.toggle('is-timeout', isTimedOut),
1417 | | (timeElement.textContent = (function formatClockTime(milliseconds) {
1418 | | const safeMilliseconds = Math.max(0, milliseconds);
1419 | | if (safeMilliseconds < 1e4)
1420 | | return `0:${(safeMilliseconds / 1e3).toFixed(1).padStart(4, '0')}`;
1421 | | const totalSeconds = Math.ceil(safeMilliseconds / 1e3);
1422 | | return `${Math.floor(totalSeconds / 60)}:${String(totalSeconds % 60).padStart(2, '0')}`;
1423 | | })(remainingMs));
1424 | | }
1425 | | function renderStatus() {
1426 | | const historyLength = game.history().length;
1427 | | ((statusElement.textContent = (function getStatusText() {
1428 | | return game.isCheckmate()
1429 | | ? ` ${colorName(oppositeColor(game.turn()))} 체크메이트 승리`
1430 | | : game.isStalemate()
1431 | | ? `스테일메이트 무승부`
1432 | | : game.isInsufficientMaterial()
1433 | | ? `기물 부족 무승부`

```

```

1434 |         : game.isThreefoldRepetition()
1435 |         ? '3회 반복 무승부'
1436 |         : game.isDrawByFiftyMoves()
1437 |         ? '50수 규칙 무승부'
1438 |         : game.isDraw()
1439 |         ? '무승부'
1440 |         : timedOutColor
1441 |         ? `${colorName(timedOutColor)} 시간 초과` : `${colorName(oppositeColor(timedOutColor))} 승리`
1442 |         : aiThinking
1443 |         ? aiProgress
1444 |         ? `AI 계산 중` : depth >= aiProgress.depthReached ? (function formatNodes(
1445 |             nodes,
1446 |             ) {
1447 |                 return nodes >= 1e6
1448 |                 ? `${(nodes / 1e6).toFixed(1)}M`
1449 |                 : nodes >= 1e3
1450 |                 ? `${Math.round(nodes / 1e3)}k`
1451 |                 : String(nodes);
1452 |             })(aiProgress.nodes) : nodes`
1453 |         : 'AI 계산 준비 중'
1454 |         : game.isCheck()
1455 |         ? `${colorName(game.turn())} 체크`
1456 |         : `${colorName(game.turn())} 차례`;
1457 |     }()),
1458 |     (turnMetaElement.textContent = colorName(game.turn())),
1459 |     (modeMetaElement.textContent =
1460 |     'local' === mode
1461 |     ? '로컬 2인'
1462 |     : `AI 상대` : 인간 : `${colorName(humanColor)} : ${function presetName(preset) {
1463 |         switch (preset) {
1464 |             case 'fast':
1465 |                 return '빠름';
1466 |             case 'balanced':
1467 |                 return '균형';
1468 |             case 'strong':
1469 |                 return '최강';
1470 |             }
1471 |         })(aiSettings.preset)}`),
1472 |     (moveMetaElement.textContent = `${game.moveNumber()}수` : `${historyLength} half-move`));
1473 | }
1474 | function getSquareClasses(square, lastMove) {
1475 |     const classes = ['board-square', 'light' === game.squareColor(square) ? 'is-light' : 'is-dark'],
1476 |     destinationMove = legalMoves.find((move) => move.to === square);
1477 |     return (
1478 |     selectedSquare === square && classes.push('is-selected'),
1479 |     destinationMove && classes.push(destinationMove.isCapture() ? 'is-capture' : 'is-legal'),
1480 |     !lastMove ||
1481 |     (lastMove.from === square && lastMove.to !== square) ||
1482 |     classes.push('is-last-move'),
1483 |     (function isCheckedKing(square) {
1484 |         const piece = game.get(square);
1485 |         return Boolean(
1486 |         piece && 'k' === piece.type && piece.color === game.turn() && game.isCheck(),
1487 |         );
1488 |     })(square) && classes.push('is-check'),
1489 |     classes.join(' ')
1490 |     );
1491 | }
1492 | function getBoardOrientation() {
1493 |     return 'ai' === mode ? humanColor : game.turn();
1494 | }
1495 | function canUseBoard() {
1496 |     return !(
1497 |     game.isGameOver() ||
1498 |     null !== timedOutColor ||
1499 |     aiThinking ||
1500 |     pendingPromotion ||
1501 |     ('local' !== mode && game.turn() !== humanColor)
1502 |     );
1503 | }
1504 | (boardElement.addEventListener('click', (event) => {

```



```

1505 |     const button = (function getSquareButton(target, boardElement) {
1506 |         if (!(target instanceof Element)) return null;
1507 |         const button = target.closest('.board-square');
1508 |         return button && boardElement.contains(button) ? button : null;
1509 |     })(event.target, boardElement);
1510 |     button &&
1511 |         (function isSquare(value) {
1512 |             return 'string' === typeof value && SQUARE_SET.has(value);
1513 |         })(button.dataset.square) &&
1514 |         (function handleSquareClick(square) {
1515 |             if (!canUseBoard()) return;
1516 |             const piece = game.get(square);
1517 |             if (selectedSquare) {
1518 |                 const movesToDestination = legalMoves.filter((move) => move.to === square);
1519 |                 return movesToDestination.length > 0
1520 |                     ? movesToDestination.some((move) => move.isPromotion())
1521 |                     ? ((pendingPromotion = {
1522 |                         from: selectedSquare,
1523 |                         to: square,
1524 |                         moves: movesToDestination,
1525 |                     }),
1526 |                       void render())
1527 |                     : void playMove({ from: selectedSquare, to: square })
1528 |                     : piece?.color === game.turn()
1529 |                     ? void selectSquare(square)
1530 |                     : (clearSelection(), void render());
1531 |             }
1532 |             piece?.color === game.turn() && selectSquare(square);
1533 |         })(button.dataset.square);
1534 |     }),
1535 |     modeButtons.forEach((button) => {
1536 |         button.addEventListener('click', () => {
1537 |             const nextMode = button.dataset.mode;
1538 |             (function isGameMode(value) {
1539 |                 return 'local' === value || 'ai' === value;
1540 |             })(nextMode) &&
1541 |                 nextMode !== mode &&
1542 |                 ((mode = nextMode), resetGame());
1543 |         });
1544 |     }),
1545 |     colorButtons.forEach((button) => {
1546 |         button.addEventListener('click', () => {
1547 |             const nextColor = button.dataset.color;
1548 |             (function isColor(value) {
1549 |                 return 'w' === value || 'b' === value;
1550 |             })(nextColor) &&
1551 |                 nextColor !== humanColor &&
1552 |                 ((humanColor = nextColor), resetGame());
1553 |         });
1554 |     }),
1555 |     presetButtons.forEach((button) => {
1556 |         button.addEventListener('click', () => {
1557 |             const preset = button.dataset.preset;
1558 |             (function isAiPreset(value) {
1559 |                 return 'fast' === value || 'balanced' === value || 'strong' === value;
1560 |             })(preset) && updateAiSettings({ ... AI_PRESETS[preset] });
1561 |         });
1562 |     }),
1563 |     clockPresetButtons.forEach((button) => {
1564 |         button.addEventListener('click', () => {
1565 |             const preset = button.dataset.clockPreset;
1566 |             (function isClockPreset(value) {
1567 |                 return (
1568 |                     '1/1' === value ||
1569 |                     '3/2' === value ||
1570 |                     '5/3' === value ||
1571 |                     '10/5' === value ||
1572 |                     '15/10' === value
1573 |                 );
1574 |             })(preset) &&
1575 |                 preset !== clockPreset &&

```

```

1576 | | | | | ((clockPreset = preset), resetGame());
1577 | | | | | });
1578 | | | | | },
1579 | | | | | aiDepthInput.addEventListener('input', () => {
1580 | | | | |   var value;
1581 | | | | |   updateAiSettings({
1582 | | | | |     ... aiSettings,
1583 | | | | |     maxDepth:
1584 | | | | |     ((value = aiDepthInput.valueAsNumber), Math.min(5, Math.max(1, Math.round(value)))),
1585 | | | | |   });
1586 | | | | | },
1587 | | | | | aiTimeInput.addEventListener('input', () => {
1588 | | | | |   updateAiSettings({ ... aiSettings, timeLimitMs: aiTimeInput.valueAsNumber });
1589 | | | | | },
1590 | | | | | aiRandomnessInput.addEventListener('input', () => {
1591 | | | | |   updateAiSettings({ ... aiSettings, randomness: aiRandomnessInput.valueAsNumber });
1592 | | | | | },
1593 | | | | | aiQuiescenceInput.addEventListener('change', () => {
1594 | | | | |   updateAiSettings({ ... aiSettings, quiescence: aiQuiescenceInput.checked });
1595 | | | | | },
1596 | | | | | newGameButton.addEventListener('click', resetGame),
1597 | | | | | undoButton.addEventListener('click', function undoLastTurn() {
1598 | | | | |   (cancelAiMove(),
1599 | | | | |   (pendingPromotion = null),
1600 | | | | |   0 !== game.history().length
1601 | | | | |   ? ('ai' === mode && game.turn() === humanColor ? (game.undo(), game.undo()) : game.undo(),
1602 | | | | |   clearSelection(),
1603 | | | | |   resetClockState(),
1604 | | | | |   restartClockForCurrentTurn(),
1605 | | | | |   render())
1606 | | | | |   : render());
1607 | | | | | },
1608 | | | | | promotionChoices.addEventListener('click', (event) => {
1609 | | | | |   const target = event.target;
1610 | | | | |   if (!target instanceof Element) return;
1611 | | | | |   const button = target.closest('[data-promotion]'),
1612 | | | | |   promotion = button ? dataset.promotion;
1613 | | | | |   (function isPromotionPiece(value) {
1614 | | | | |     return 'q' === value || 'r' === value || 'b' === value || 'n' === value;
1615 | | | | |   })(promotion) &&
1616 | | | | |   (function finishPromotion(promotion) {
1617 | | | | |     if (!pendingPromotion) return;
1618 | | | | |     if (!pendingPromotion.moves.find((move) => move.promotion === promotion)) return;
1619 | | | | |     const { from: from, to: to } = pendingPromotion;
1620 | | | | |     ((pendingPromotion = null), playMove({ from: from, to: to, promotion: promotion }));
1621 | | | | |   })(promotion);
1622 | | | | | },
1623 | | | | | resetClockState(),
1624 | | | | | restartClockForCurrentTurn(),
1625 | | | | | render());
1626 | | | | | })();
1627 |

```

==== index.html (366 B / 3a75234ac1db) ====

```

1 | <!doctype html>
2 | <html lang="ko">
3 |   <head>
4 |     <meta charset="UTF-8" />
5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 |     <title>TS Chess</title>
7 |     <script type="module" crossorigin src="/assets/index-CSn2cQoE.js"></script>
8 |     <link rel="stylesheet" crossorigin href="/assets/index-BVkn0d8S.css" />
9 |   </head>
10 |   <body></body>
11 | </html>
12 |

```